Pre-requisites for Servlet and JSP

1. Basic Java Programming:

- **Syntax and Semantics:** Understanding Java syntax, control structures (if, for, while), and data types.
- **Object-Oriented Programming (OOP):** Familiarity with core OOP concepts like classes, objects, inheritance, polymorphism, abstraction, and encapsulation.
- **Exception Handling:** Ability to handle exceptions using try-catch blocks and custom exceptions.
- **Collections Framework:** Knowledge of common Java collections like List, Set, and Map.

2. Web Technologies:

- **HTML:** Basic knowledge of HyperText Markup Language to structure web pages.
- **CSS:** Understanding of Cascading Style Sheets for styling web pages.
- **JavaScript:** Basic scripting knowledge to add interactivity to web pages.

3. SQL, Database, and JDBC

- **SQL:** Ability to write basic SQL queries to interact with databases.
- **Database Management:** Understanding of relational databases like MySQL or PostgreSQL, including how to connect and perform CRUD (Create, Read, Update, Delete) operations.
- **JDBC:** Know how to use JDBC with Java.

Fundamentals of Client-Server Architecture

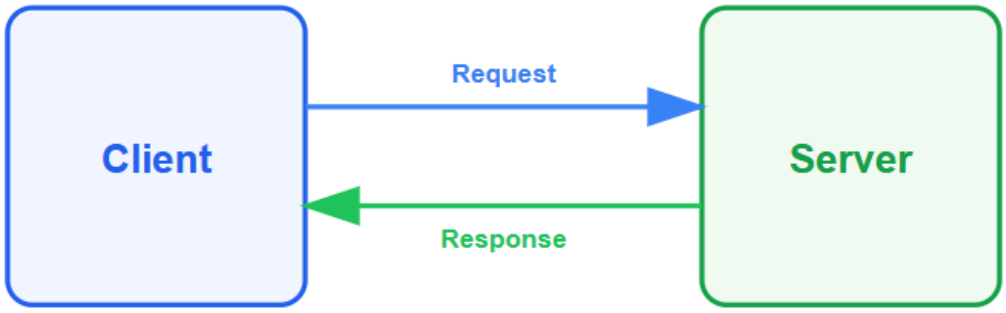
Questions?

1. ***How do you develop a web application?***
2. ***What is a servlet?***
3. ***Why do we have to use a servlet?***

👉 How do you develop a web application?

Developing a web application involves the following key steps:

1. **Client and Server Setup:**
    - **Client:** The client is a user or system that requests a specific resource or service from another entity known as the server.
    - **Server:** A server processes requests from clients. It can handle multiple requests simultaneously, ensuring efficient resource allocation and response.



2. **Internet and Web Server:**

- **Internet:** The internet is a global communication system that connects thousands of individual networks. It enables the exchange of information between multiple computers on a network.
- **Web Server:** A web server is a software program or server computer designed to offer World Wide Web access. It handles requests from clients and serves the requested web pages or resources.

3. **HTTP Protocol:**
    - **HTTP (Hypertext Transfer Protocol): T**his protocol is essential for communication between the web browser (client) and the web server. The client sends an HTTP request to the server, and the server responds with the requested resource, usually a web page.

👉 What is the need for a servlet?

Servlets handle the processing of client requests, generate dynamic content, interact with databases, and manage server-side processing to fulfill client requests.

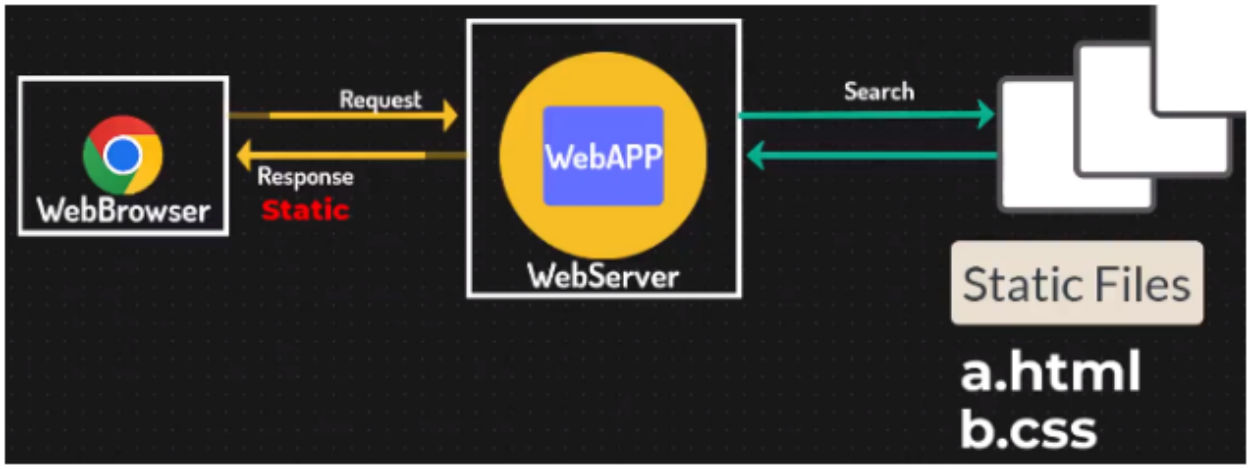
👉 Why do we have to use a servlet?

Servlets are necessary for efficiently processing multiple client requests and generating dynamic web content. They help in handling server-side business logic and database interactions, ensuring smooth and dynamic responses to client requests.

Static Response and Dynamic Response

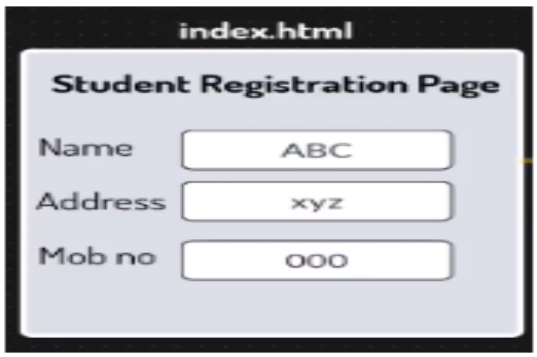
👉 Static Response:

- **Definition:** No change in the output for every user.
- **Process:**
  - The web browser sends a request to the web server.
  - The web server returns static files (e.g., HTML, CSS).
  - The response is the same for all users.



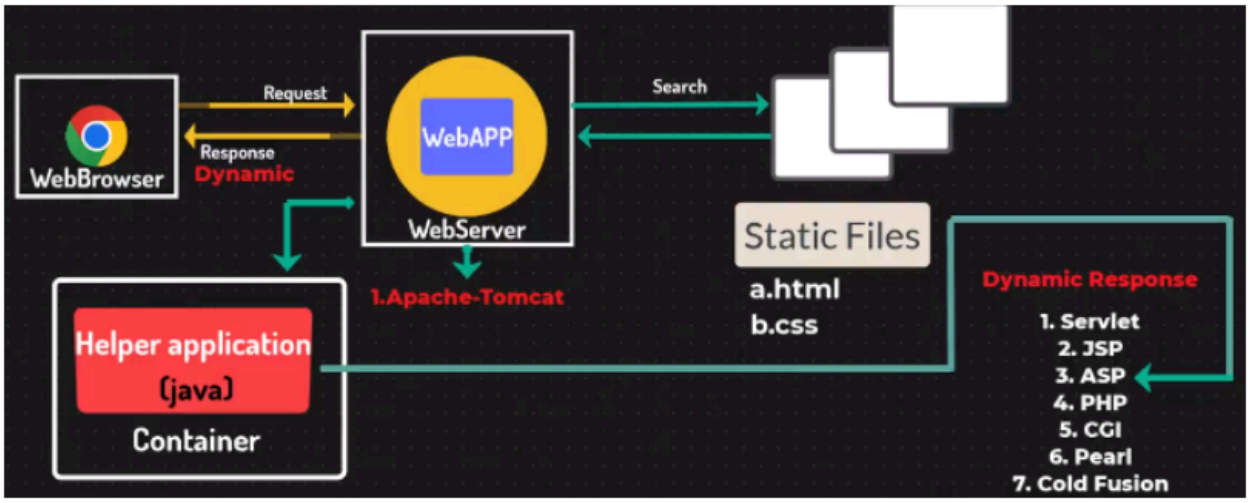
example:

- User requests a static HTML page.
- Web server serves the static HTML and CSS files.
- Response remains unchanged for all users.



👉 Dynamic Response:

- **Definition:** Output changes based on user requests.
- **Process:**
    - The web browser sends a request to the web server.
    - The web server interacts with a helper application (e.g., a Java servlet) within a container (e.g., Apache Tomcat).
    - The web server returns dynamic content generated based on the user's request.
    - Examples of technologies used for dynamic responses: Servlet, JSP, ASP, PHP, CGI, Perl, ColdFusion.



example:

- User fills out a registration form and submits it.
- Web server receives the request and forwards it to a servlet.
- The servlet processes the request (e.g., storing user details in a database).
- The servlet generates a dynamic response based on the processed data and sends it back to the web server.
- Web server returns the dynamic response to the user's browser.

Introduction to Servlets

👉 What are servlets?

- **Definition:** Servlets are server-side Java programs that handle client requests and generate dynamic web content. They run within a servlet container managed by a web server.
- **Execution:** Unlike standard Java programs, servlets are not executed by the Java Virtual Machine (JVM) directly and thus do not require a main method.

👉How Servlets Work:

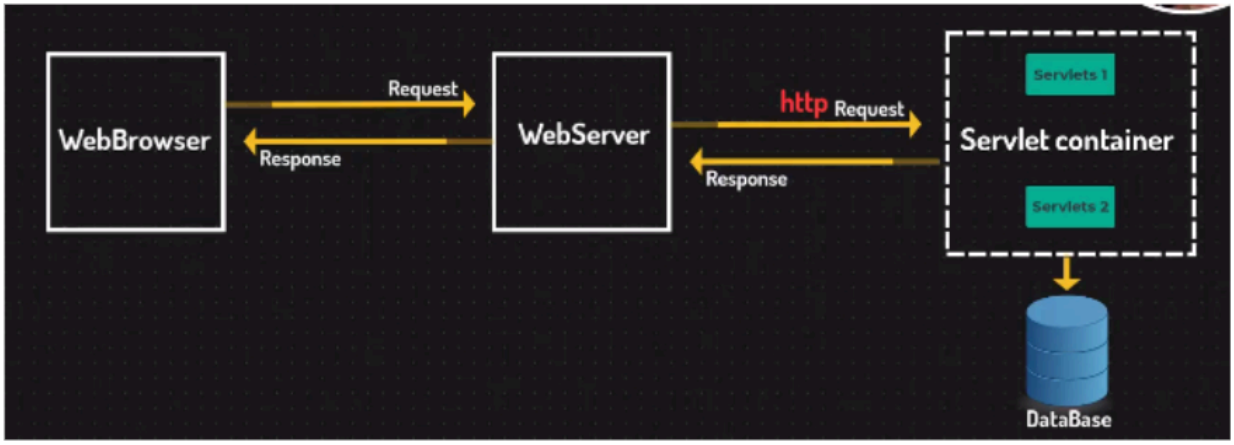
1. **Request Flow:**
    - The client (web browser) sends a request to the web server.
    - The web server forwards the request to the servlet container.
    - The servlet container locates the appropriate servlet to handle the request.
    - The servlet processes the request, potentially interacting with a database.
    - The servlet generates a response and sends it back to the web server.
    - The web server sends the response to the client's web browser.
2. **Servlet Container:**
    - A servlet container, such as Apache Tomcat, is a part of a web server that interacts with the servlets.
    - It is responsible for managing the lifecycle of servlets, mapping requests to servlets, and ensuring proper request handling.

👉 Key Points:

- **No Main Method:** Since servlets are not standalone applications, they do not need a main method. Instead, they are managed by the servlet container.
- **Server Software:** To run servlets, you need server software like Apache Tomcat that can host the servlet container and handle HTTP requests.

👉 Setting Up a Server:

- **Server Software:** Install server software (e.g., Apache Tomcat).
- **Run Servlet Container:** Ensure the servlet container is running to process incoming HTTP requests and handle servlet execution.



Install Apache Tomcat Server

👉Download Apache Tomcat:

- Go to the Apache Tomcat download page.
- Choose the version you want to download. Typically, you would download the latest stable version.
- Download the 32-bit/64-bit Windows Service Installer.

For Windows users:

- Download the installer at https://dlcdn.apache.org/tomcat/tomcat-9/v9.0.93/bin/apache-tomcat-9.0.93.exe

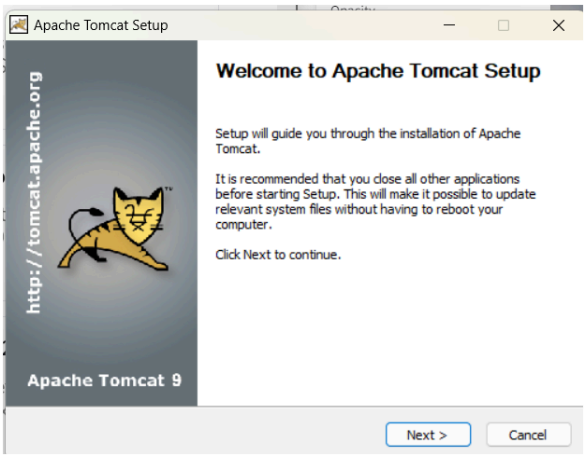
For Mac and Linux Users:

- https://downloads.apache.org/tomcat/tomcat-9/v9.0.93/bin/apache-tomcat-9.0.93.tar.gz.sha512

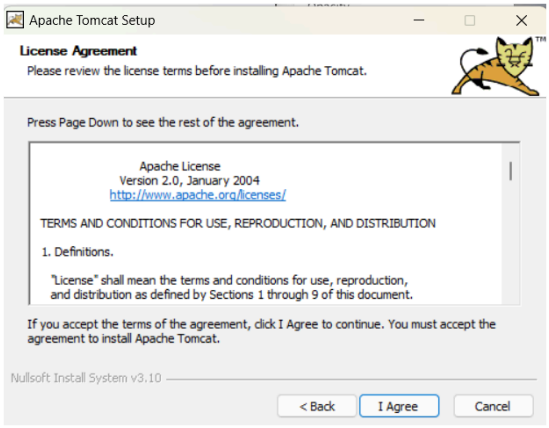
👉Installation:

Step 1: Double-click on the exe file.

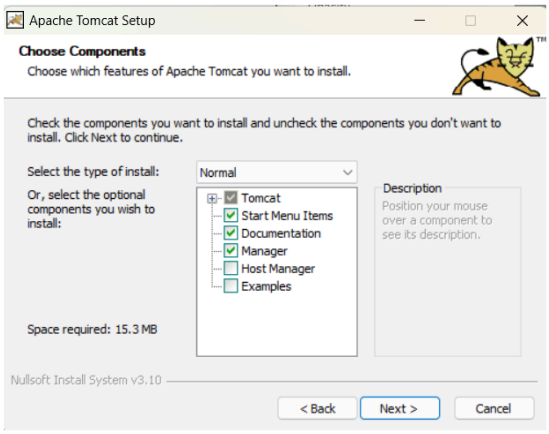
Step 2: Click next.



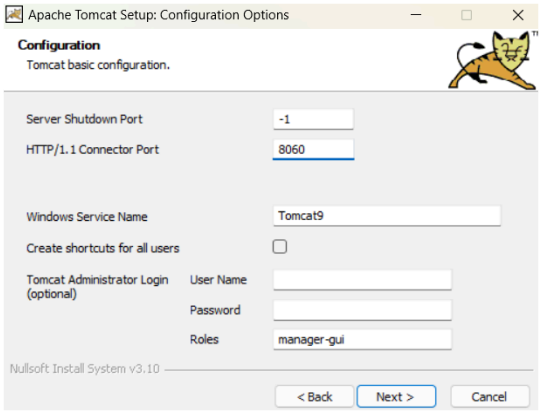
Step 3: Agree to the agreement



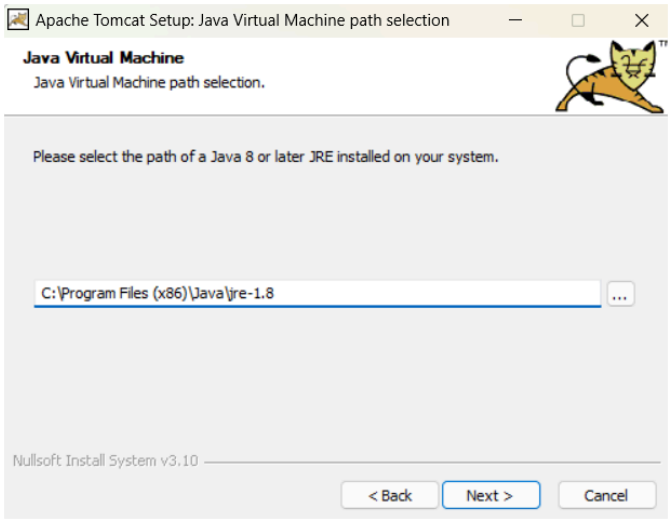
Step 4: Remain default and click next.

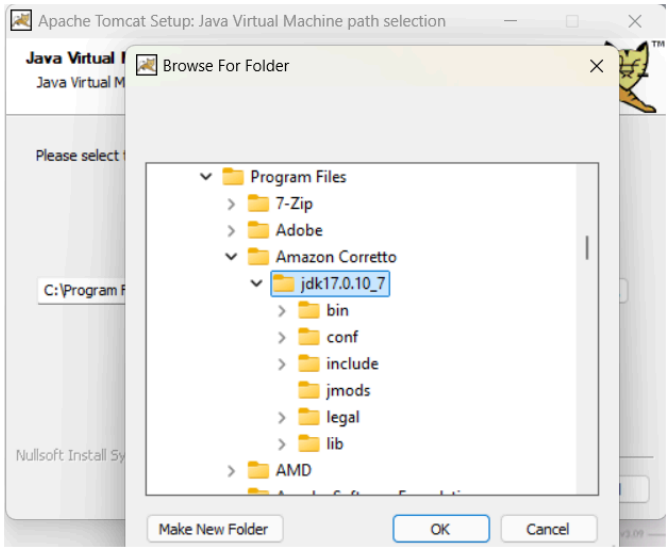


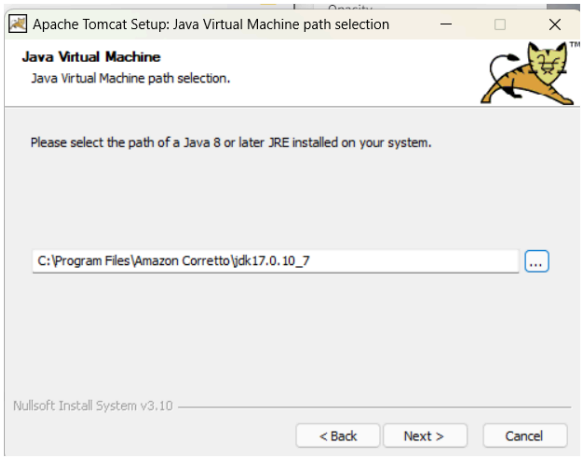
Step 5: Click next after changing the port from the default 8080 if you would like. This is because 8080 is a very common port number, and there is a good chance that many different applications are using it.



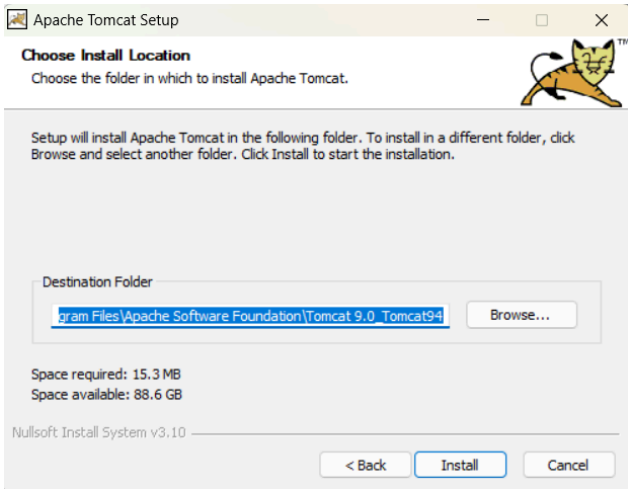
Step 6: Default jre path; now you can make changes.



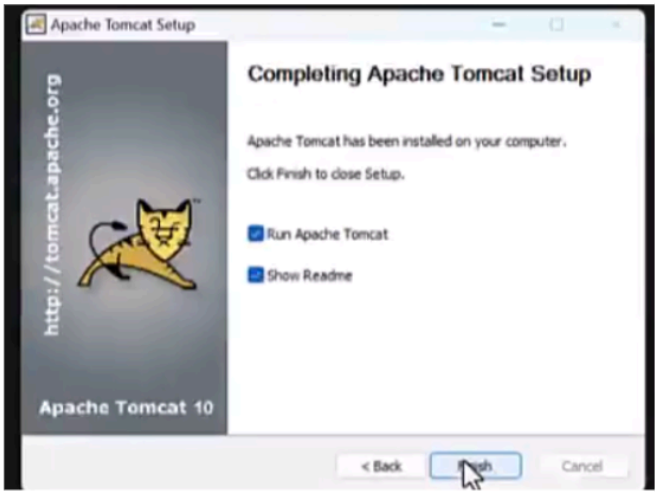




Step 7: Click on Install

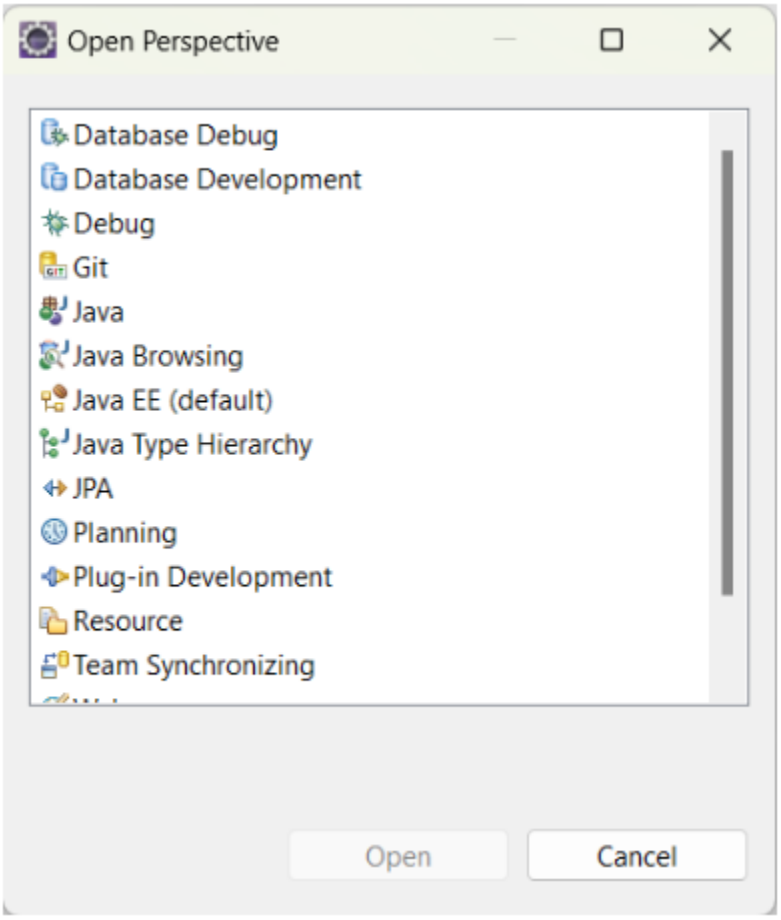


Step 8: Make finish


Apache Tomcat Setup
http://tomcat.apache.org
Apache Tomcat 10
Completing Apache Tomcat Setup
Apache Tomcat has been installed on your computer.
Click Finish to close Setup.
Run Apache Tomcat
Show Readme
< Back
Finish
Cancel

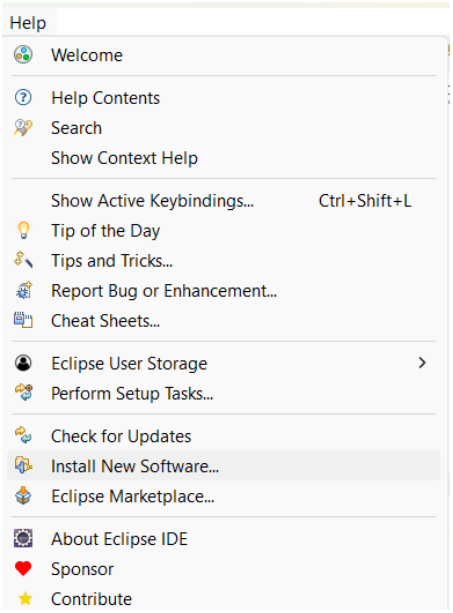
Configure Apache Tomcat in Eclipse IDE

Choose perspective as Java EE:



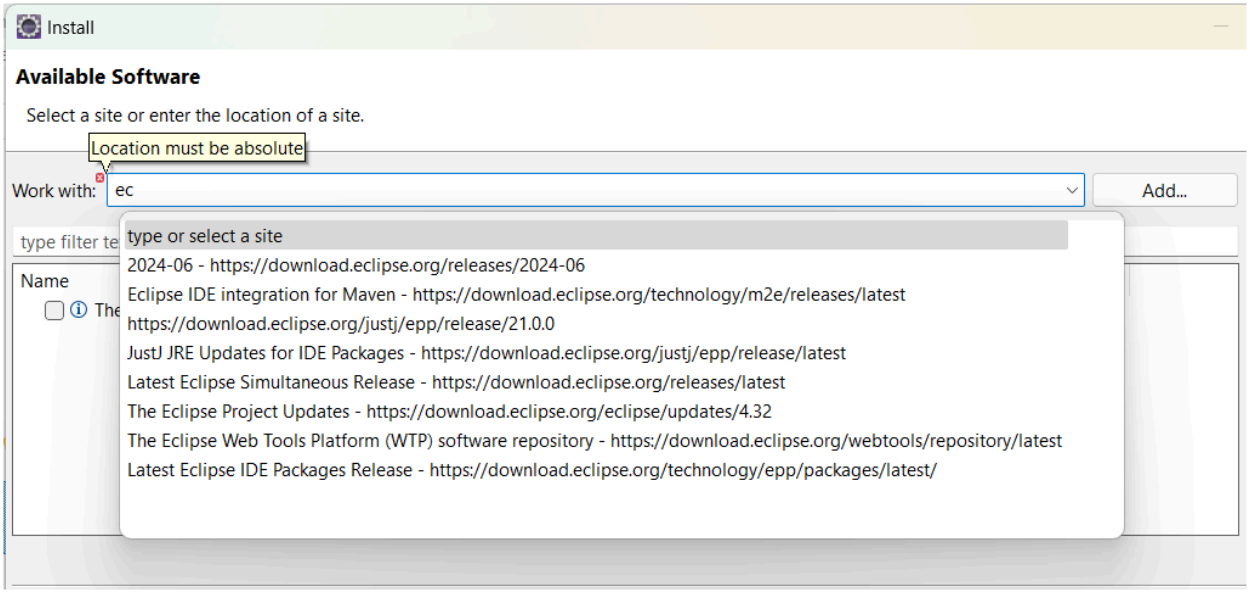
👉 **Note: If you do not have Java EE (two ways to setup)**

1. Uninstall Eclipse and download new one from eclipse.
2. I. Go to help



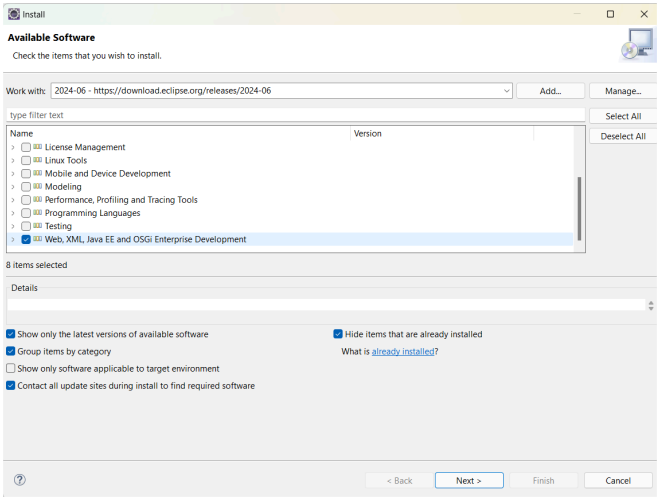
II. Install New Software

(https://download.eclipse.org/release/2024-06)



III. Now select

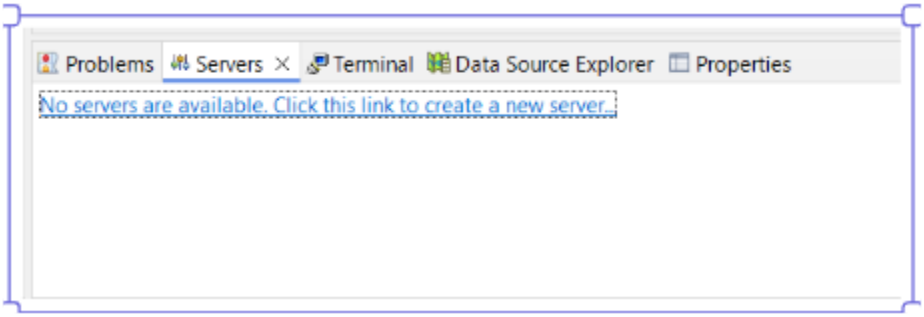
(Web XML, Java EE option)



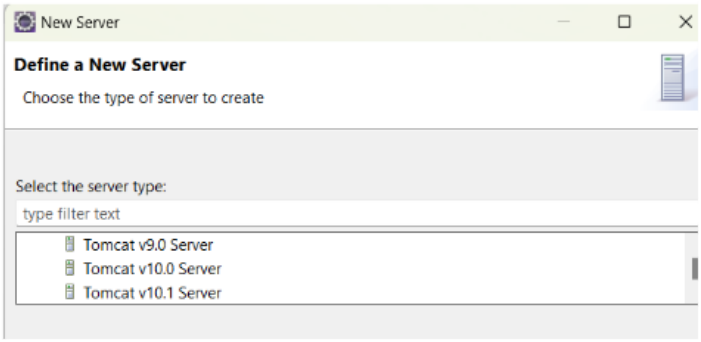
IV. Restart the application

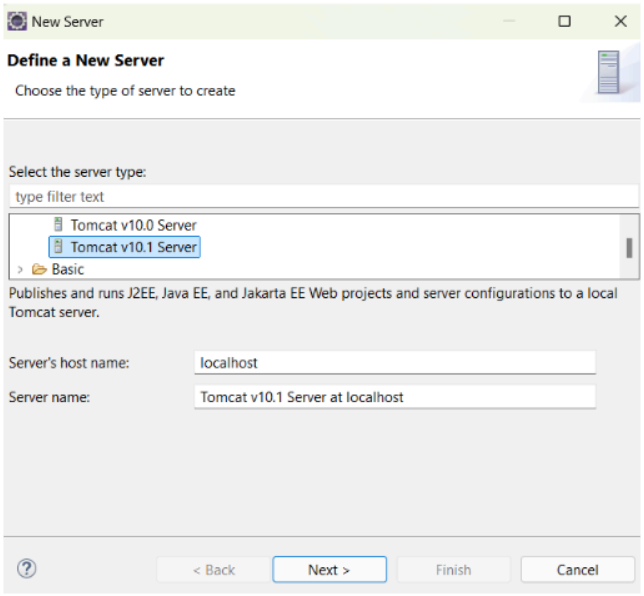
👉 Setup Server:

Step 1: Click on Create a new server.

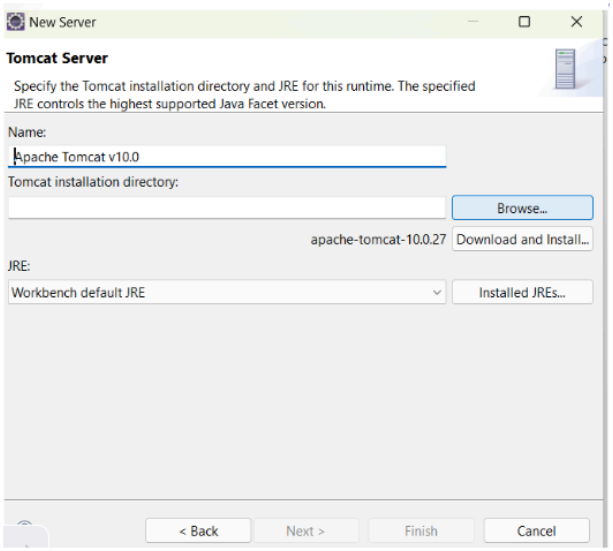


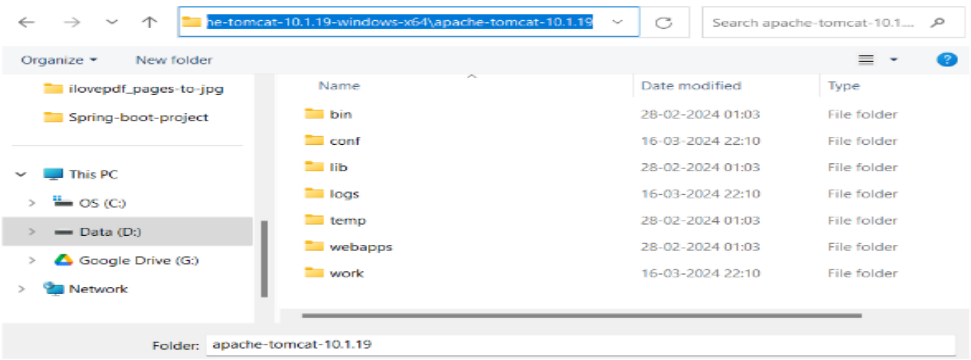
Step2: Define a New Server



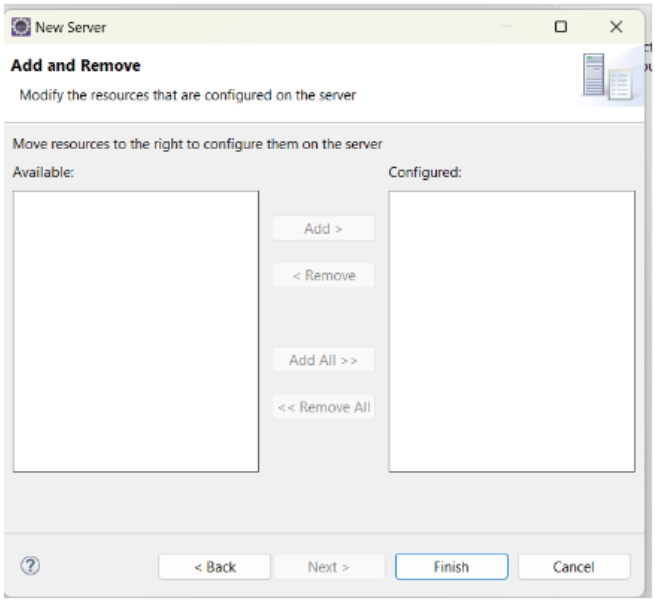


Step 3: Browse Path for tomcat Server

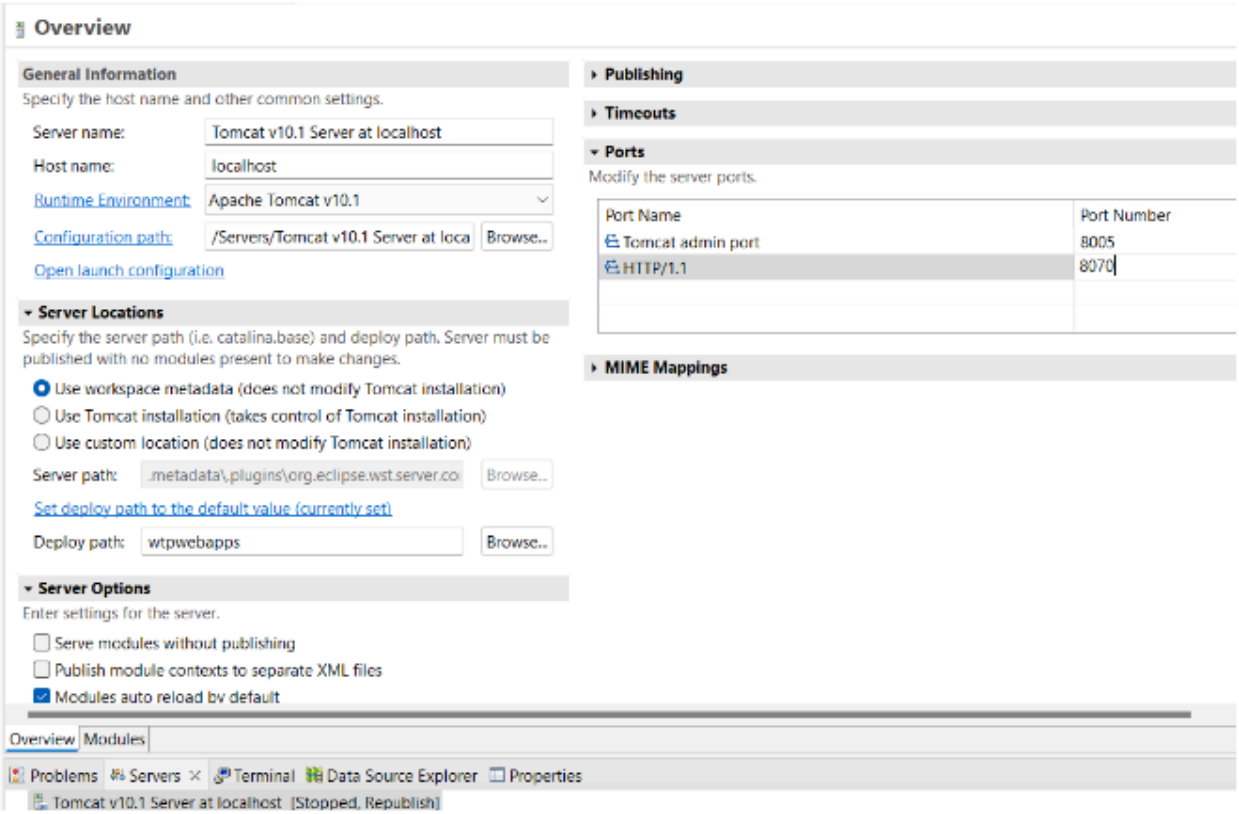




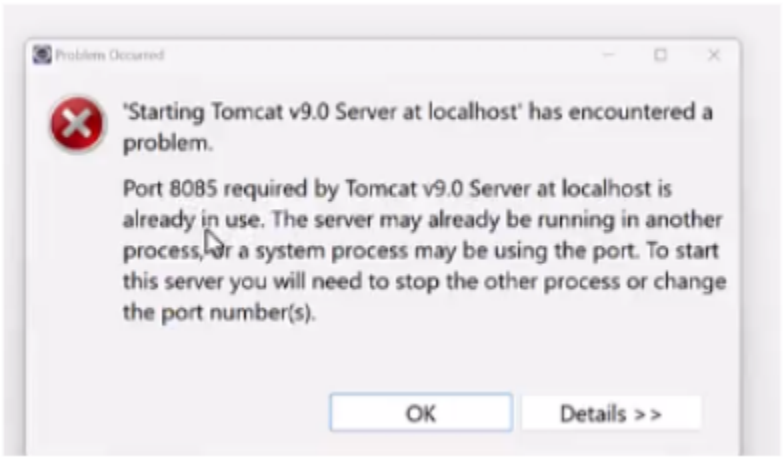
Step 5: Make finish



Step 6: Double-click on server to see all configuration settings.



Sometimes an error occurs:

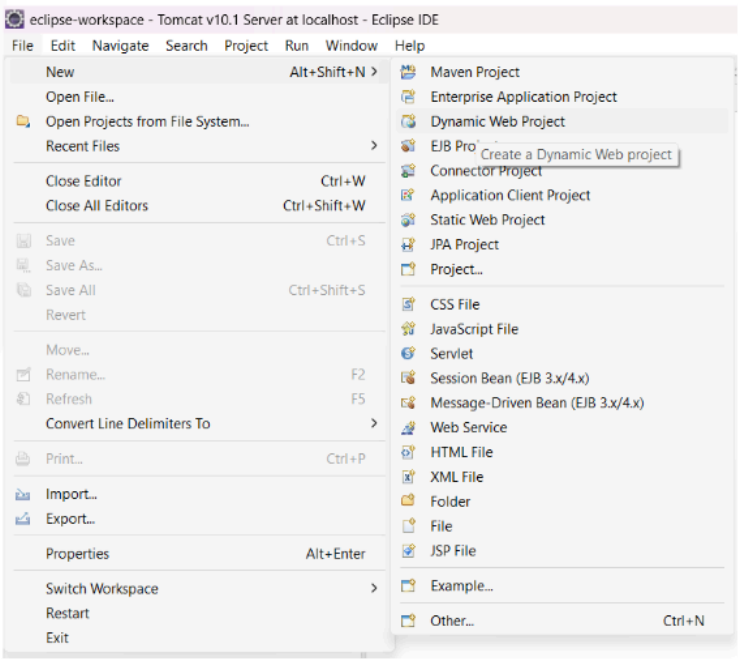


- Just change the port in the configuration.

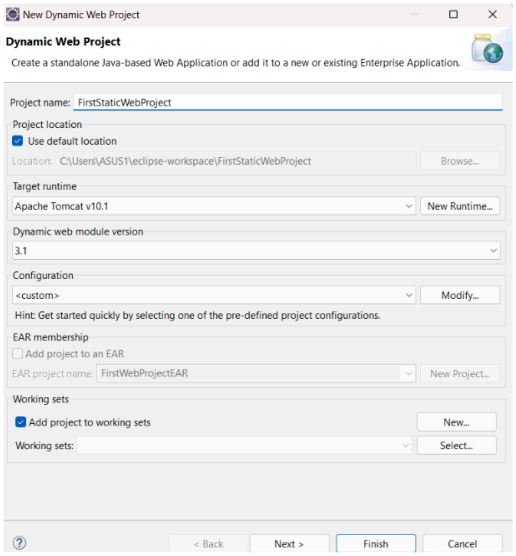
First Web App with Static Response

👉 Setup project:

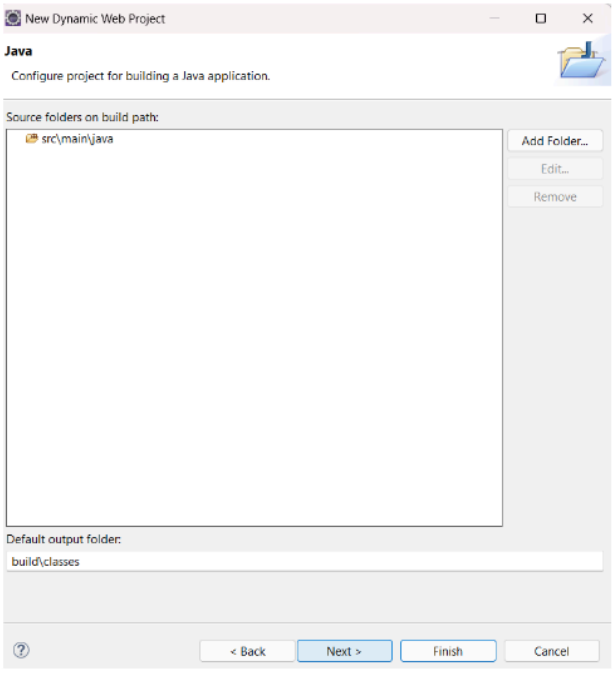
Step 1: Go to the new file and select Dynamic Web Project.



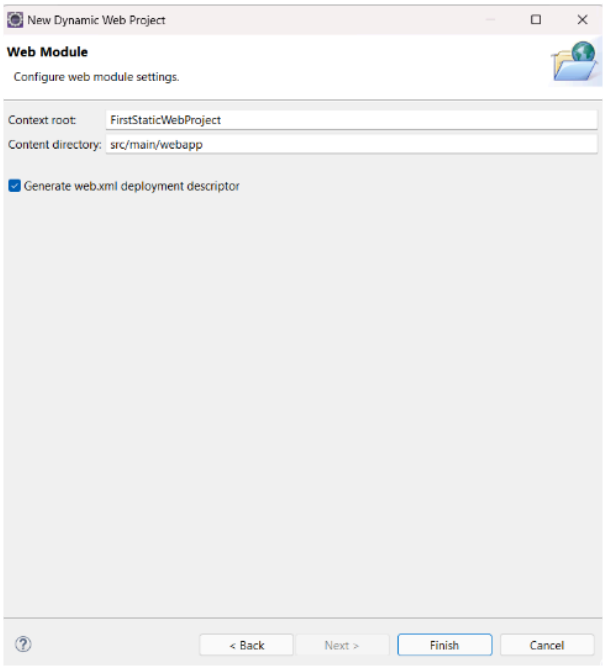
Step 2: Add project name, target runtime, and dynamic web module version.



Step 3: click next.

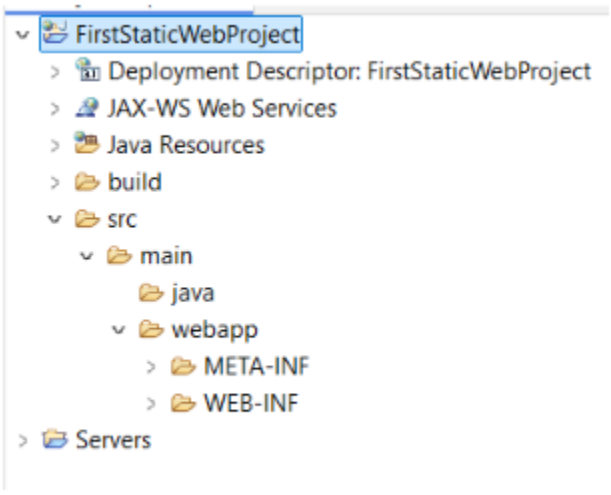


Step 4: Make finish



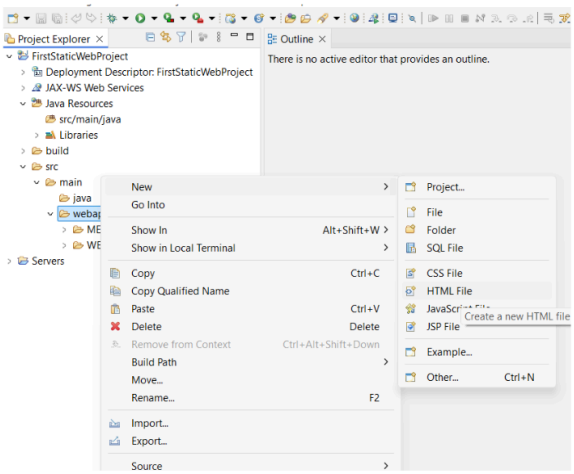
Note: If you want to configure your servlet using web.xml, make sure to check the "Generate web.xml" option during installation. If you prefer to use annotations for configuration, leave this option unchecked.

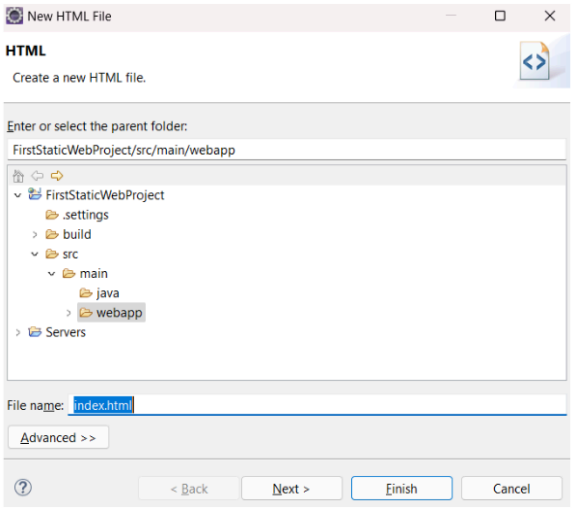
👉Project Structure:



- Inside the Java folder, we can write Java code.
- Inside the webapp folder we can write HTML and JSP.

👉Create the first index.html file:





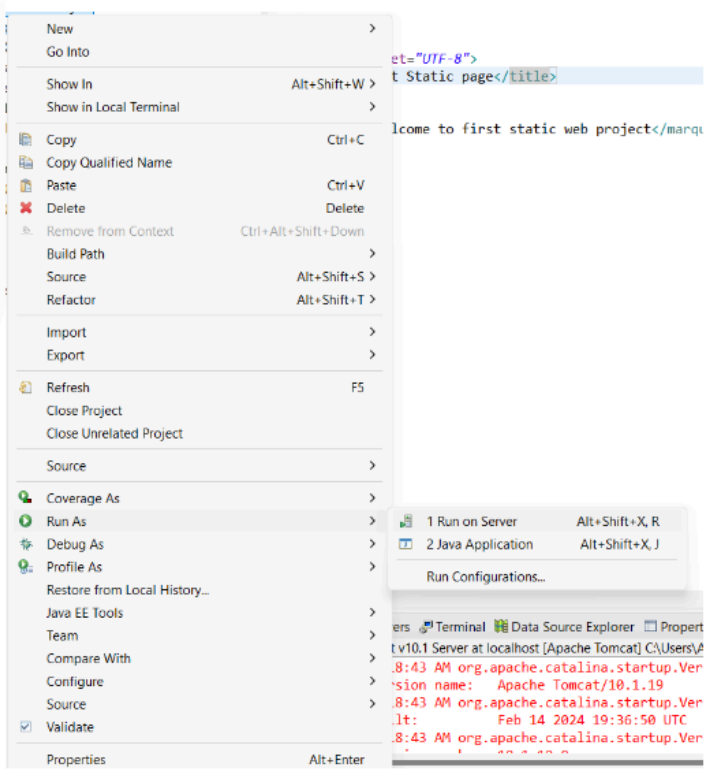
index.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
```

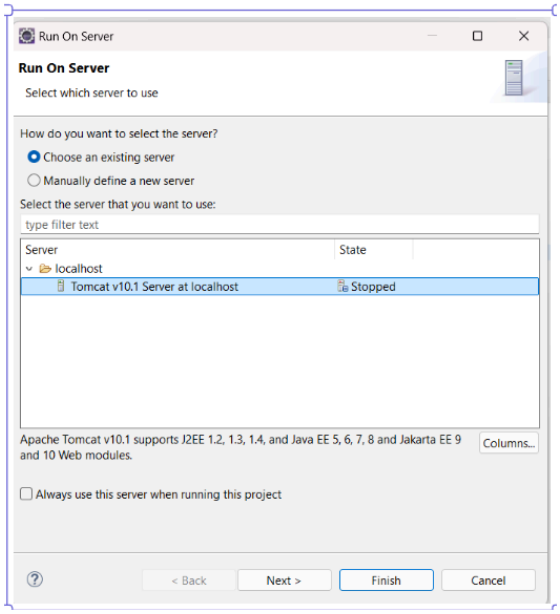
```
<title>Insert title here</title>
</head>
<body>
<marquee>Welcome to first static web project</marquee>
</body>
</html>
```

👉Run your first web app:

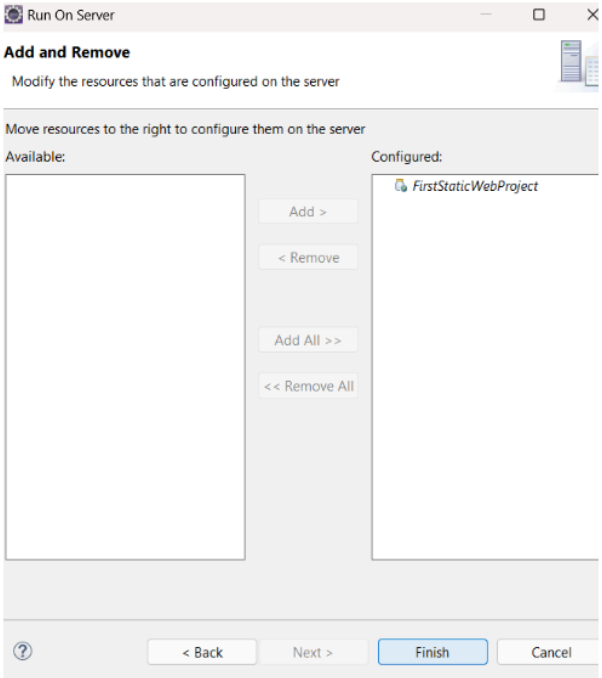
Step 1: Click on Run As and choose Run on Server.



Step 2: Choose Server



Step 3: Add and remove project from the server



Click Finish.

Result:

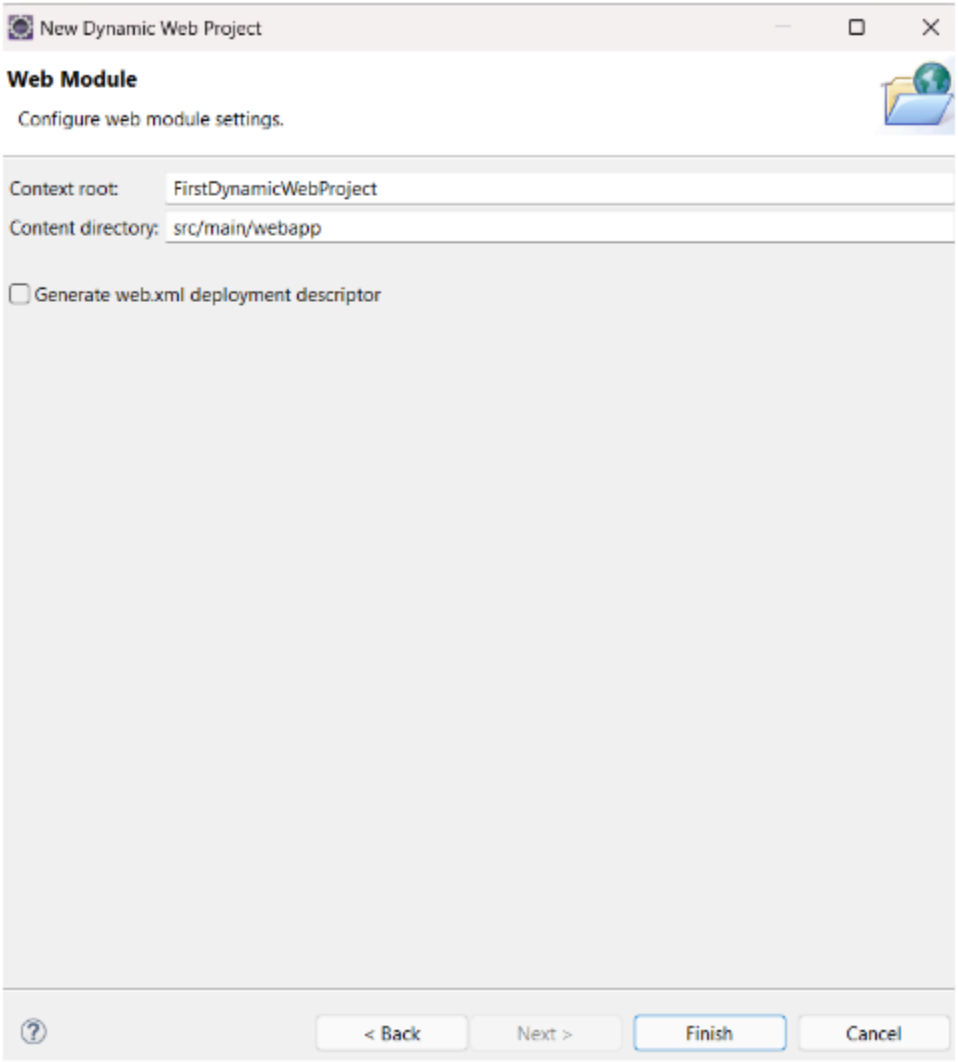
output:

First Servlet WebApp with Dynamic Response

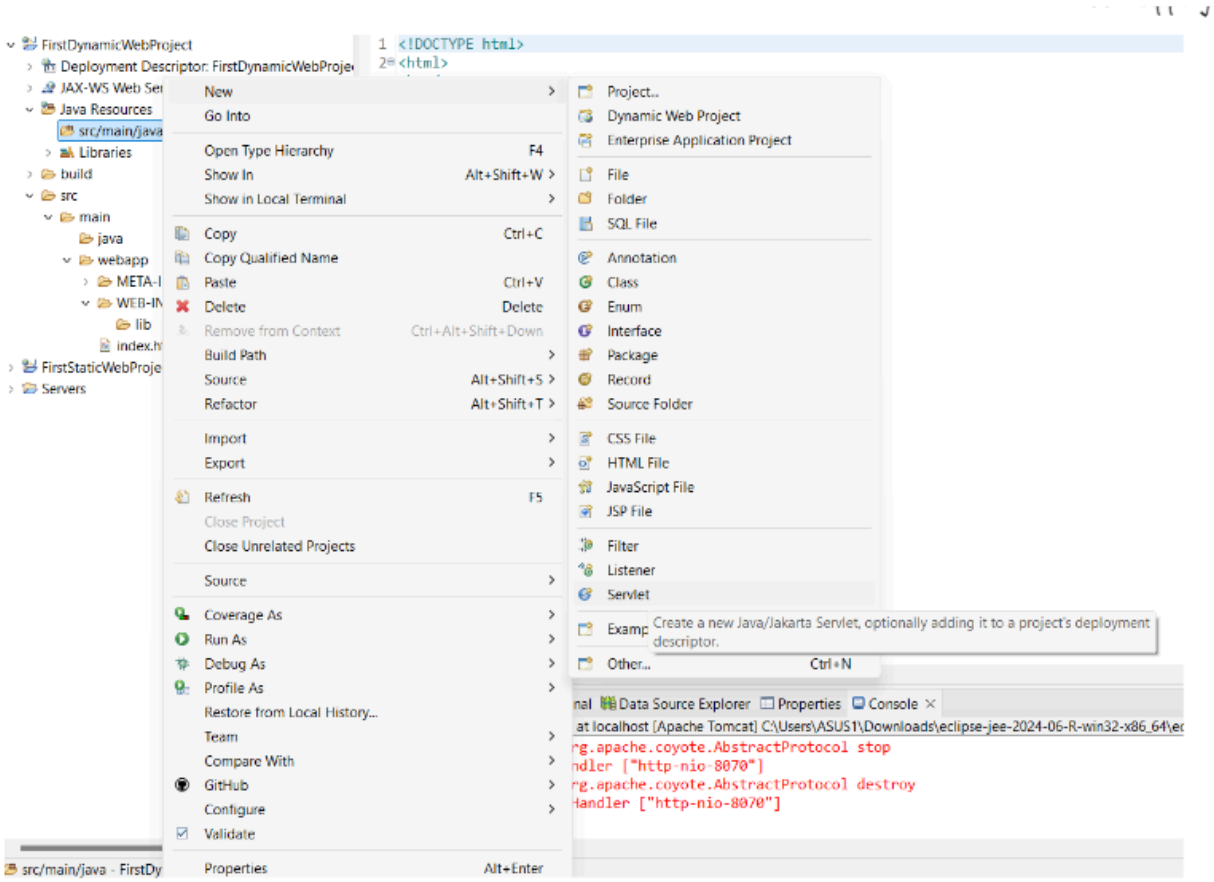
Note: If you do not stop the server and try to run another application, it will show an error that the port is busy.

👉Create First WebApp with Servlet

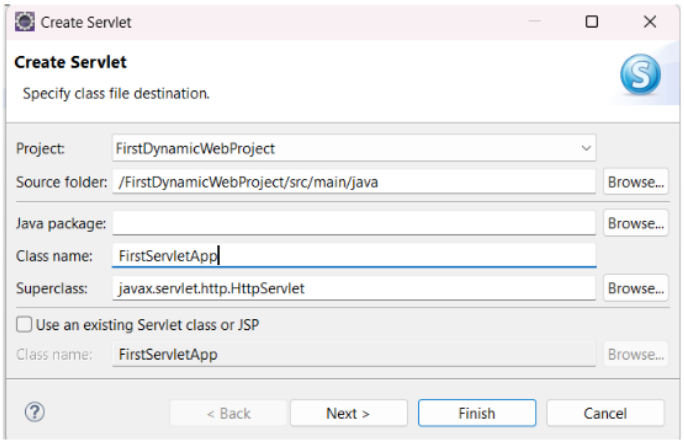
Step 1: Create a web app project without *web.xml*.



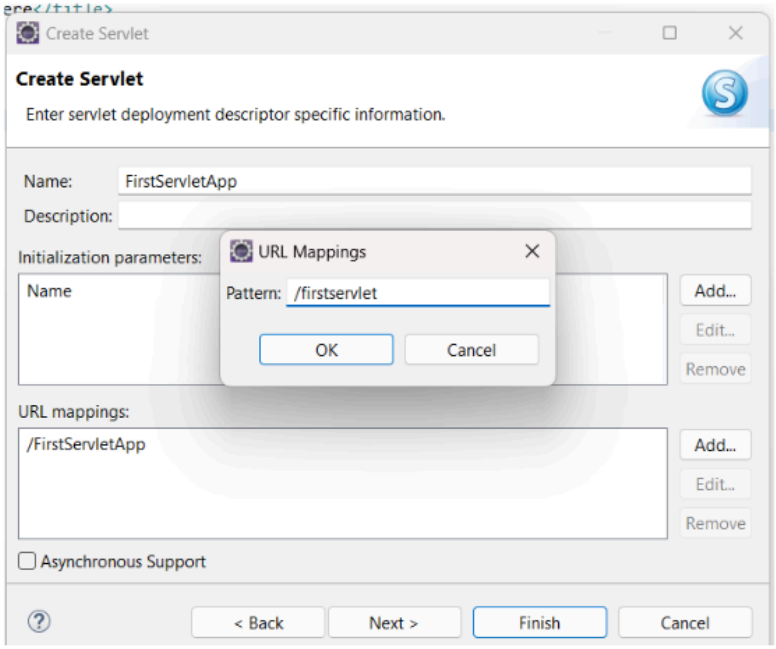
Step 2: In src/main/java, create a servlet inside a package or in the default package.



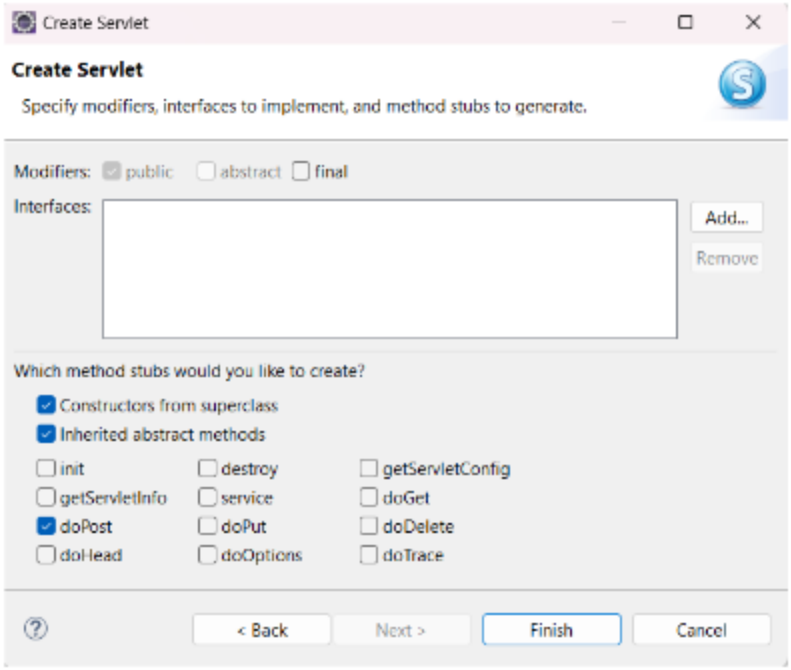
- Add class name



- Add routing path

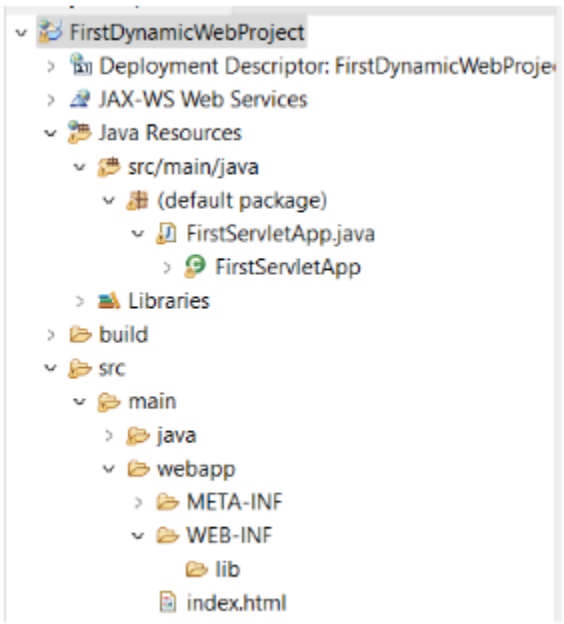


- For the first servlet project, select only this option.



Make finish.

👉Project Structure:



Code:

Index.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body style="font-family: Arial, sans-serif; background-color: #f4f4f9;
display: flex; justify-content: center; align-items: center; height:
100vh; margin: 0;">
  <form style="background-color: #ffffff; padding: 20px;
border-radius: 8px; box-shadow: 0 0 10px rgba(0, 0, 0, 0.1); width:
300px;"  action="./firstservlet" method="post">
      <h2 style="text-align: center; color: #333333;">User
Information</h2>

      <div style="margin-bottom: 15px;">
          <label for="uname" style="display: block; font-weight: bold;
margin-bottom: 5px; color: #555555;">Name:</label>
          <input type="text" id="uname" name="uname" style="width:
100%; padding: 10px; border: 1px solid #cccccc; border-radius: 4px;">
      </div>
      <div style="margin-bottom: 20px;">
          <label for="ucity" style="display: block; font-weight: bold;
margin-bottom: 5px; color: #555555;">City:</label>
          <input type="text" id="ucity" name="ucity" style="width:
100%; padding: 10px; border: 1px solid #cccccc; border-radius: 4px;">
```

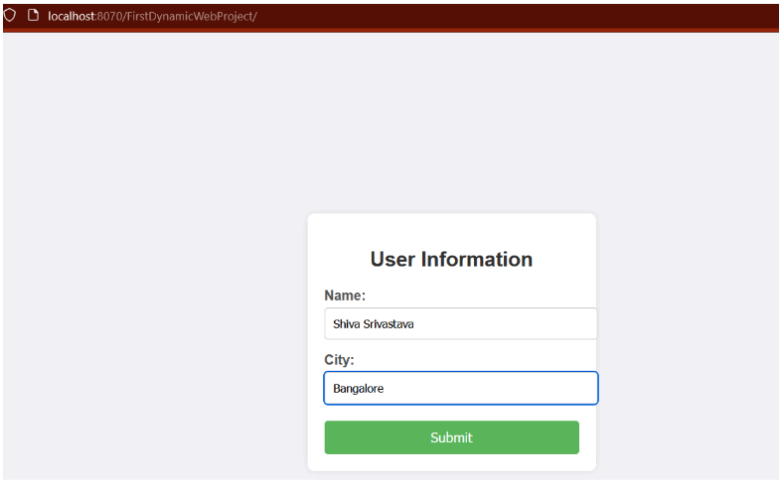
```
    </div>
    <input type="submit" value="Submit" style="width: 100%;
padding: 10px; background-color: #5cb85c; color: white; border: none;
border-radius: 4px; font-size: 16px; cursor: pointer;">
  </form>
</body>
</html>
```

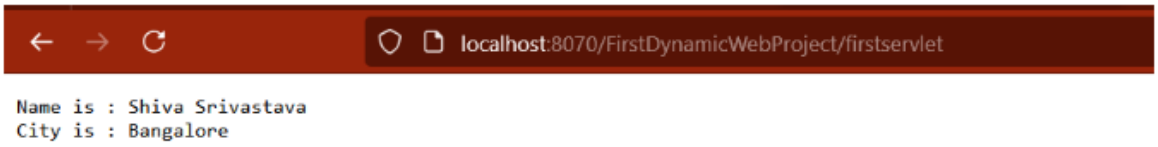
FirstServletApp.java

```
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
@WebServlet("/firstservlet")
public class FirstServletApp extends HttpServlet {

  public FirstServletApp() {
    System.out.println("Internally constructor called");
  }

      public void doPost(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException
{

            String name=request.getParameter("uname");
```

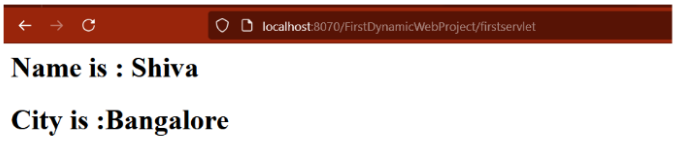
```
            String city=request.getParameter("ucity");

            PrintWriter pw=response.getWriter();
            pw.println("Name is : "+name);
            pw.println("City is : "+city);

            //use html tag for formating
//          pw.println("<h1> Name is : "+name+"</h1>");
//          pw.println("<h1> City is :"+city+"</h1>");

            pw.close();

        }
}
```

Output:





Uncomment the HTML tag.



👉Servlet Concepts and Annotations

@WebServlet Annotation

The @WebServlet annotation is used to declare a servlet.

Example:

```
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
@WebServlet("/firstservlet")
public class FirstServletApp extends HttpServlet {

}
```

Request and Response Objects

- **HttpServletRequest**: Represents the client's request. It contains information such as request parameters, headers, and attributes.
    - Methods:
        - getParameter(String name): Retrieves the value of a request parameter.
        - getParameter(String city): Retrieves the value of a request parameter.

```
String name=request.getParameter("uname");
String city=request.getParameter("ucity");
```

- **HttpServletResponse**: Represents the server's response. It allows you to set the response status, headers, and content.
    - Methods:
        - getWriter(): Returns a PrintWriter object to send character text to the client.

```
PrintWriter out = response.getWriter();
```

HttpServlet

- HttpServlet is a class provided by Java EE that extends GenericServlet and provides methods to handle HTTP requests.
- To create a servlet, extend the HttpServlet class.
- Override methods like doGet, doPost, doPut, etc., to handle specific types of HTTP requests.

```
public class FirstServletApp extends HttpServlet {
```

```
        @Override
        protected void doGet(HttpServletRequest request,
HttpServletResponse response) throws IOException {
        // Handle GET request
        }
}
```

Constructor in Servlets

- The constructor of a servlet is called when the servlet is first loaded into memory.

```
public class FirstServletApp extends HttpServlet {
        public FirstServletApp () {
        // Constructor code
        }
}
```

PrintWriter

- PrintWriter is used to send text data to the client.
- It is obtained from the HttpServletResponse object.
- Example:

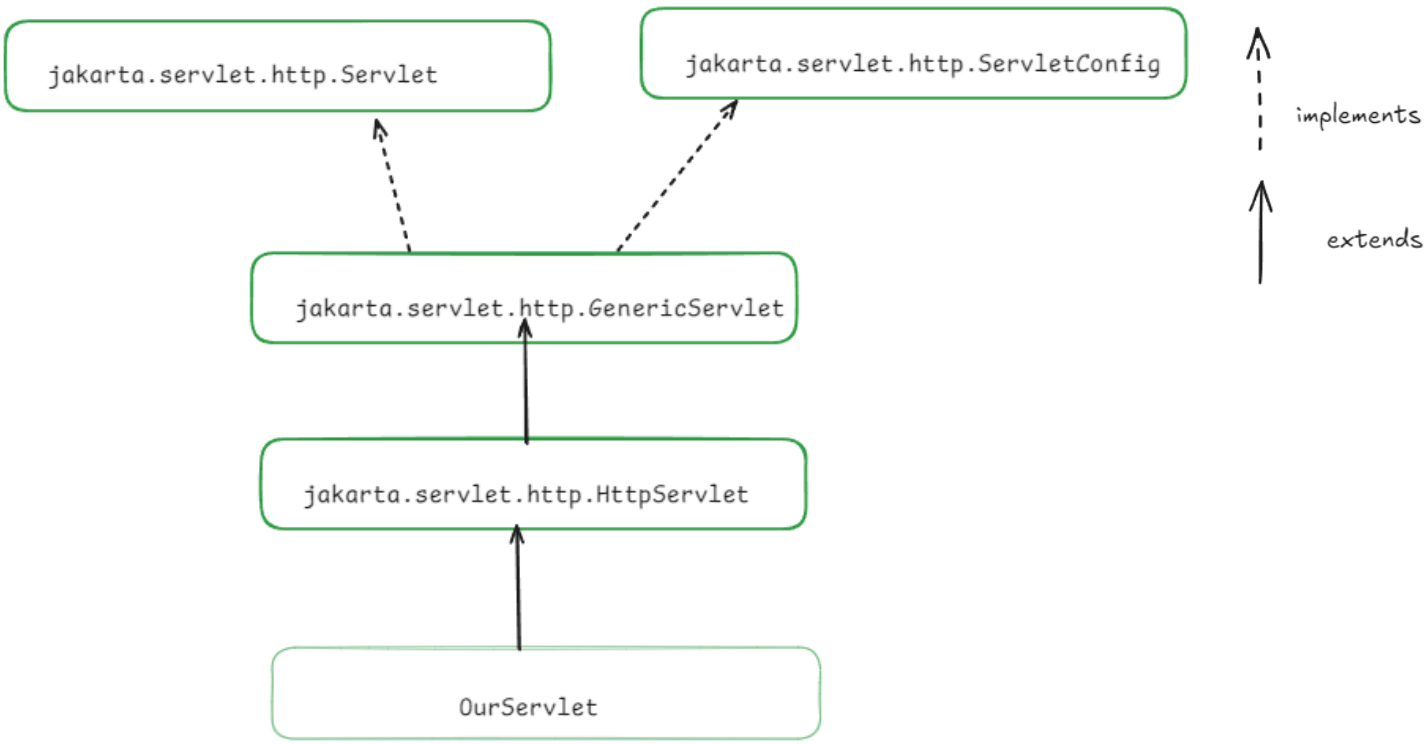
```
public void doPost(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
                        String name=request.getParameter("uname");
                        String city=request.getParameter("ucity");

    PrintWriter pw=response.getWriter();
```

```
  pw.println("Name is : "+name);
   pw.println("City is : "+city);

    //use html tag for formating
//       pw.println("<h1> Name is : "+name+"</h1>");
//   pw.println("<h1> City is :"+city+"</h1>");

     pw.close();

                }
```

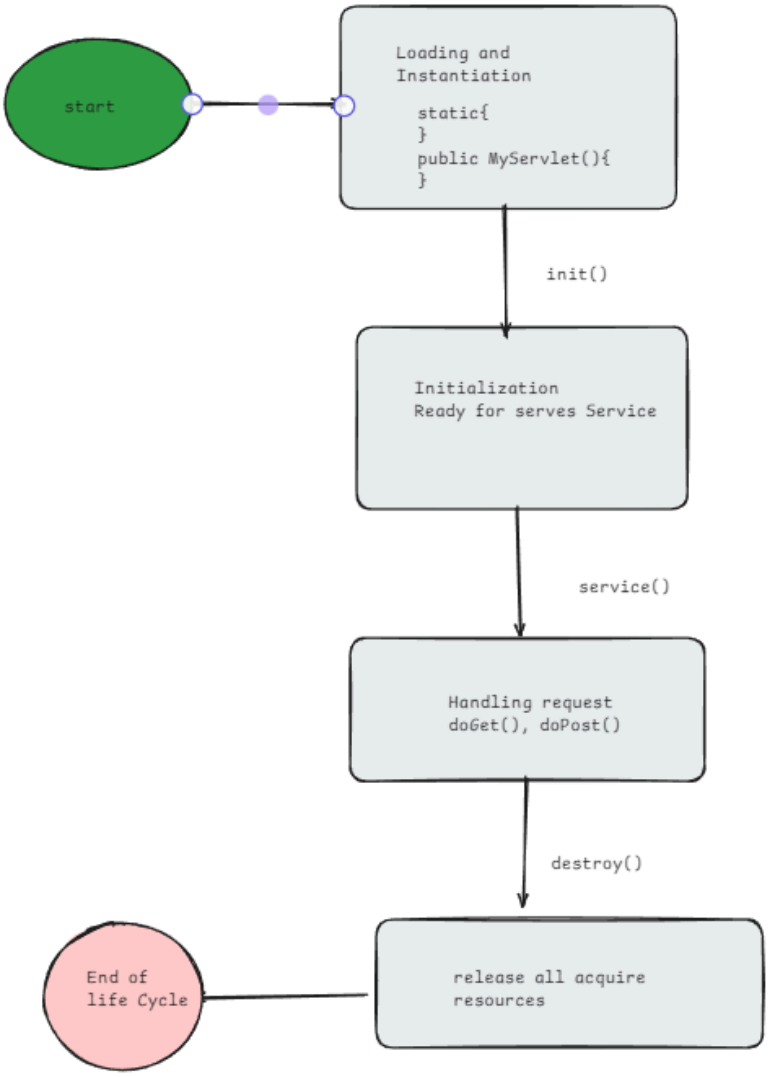
Servlet Lifecycle

The servlet lifecycle is the process through which a servlet goes from creation to destruction in a web application. Understanding this lifecycle is crucial for effective servlet management. Here are the key stages:

👉Hierarchy of Servlet



👉 Stages of Servlet lifecycle



1. Loading and Instantiation

- **Loading**: The servlet container (e.g., Tomcat) loads the servlet class when it is first requested or at server startup if it is configured for loading at startup.
- **Instantiation**: The container creates an instance of the servlet class using the default constructor.

2. Initialization (init Method)

- **Purpose**: The init() method is called by the servlet container after instantiation and is used for initializing the servlet.
- **Signature**: public void init(ServletConfig config) throws ServletException
- **Initialization Parameters**: You can access initialization parameters via the ServletConfig object passed to the init() method.
- **Note**: The init() method is called only once during the lifecycle of the servlet.

3. Request Handling (service Method)

- **Purpose**: The service() method is called by the container each time a request is made to the servlet.
- **Signature**: public void service(ServletRequest req, ServletResponse res) throws ServletException, IOException
- **Process**:
    - The service() method determines the type of request (e.g., GET, POST) and dispatches it to the appropriate handler method (doGet(), doPost(), etc.).
    - **Multithreading**: The service() method can be called concurrently for multiple requests, so servlets must be thread-safe.
- **Common Methods**:

    - doGet(HttpServletRequest req, HttpServletResponse res)
    - doPost(HttpServletRequest req, HttpServletResponse res)

4. Destruction (destroy Method)

- **Purpose**: The destroy() method is called by the servlet container to indicate that the servlet is being taken out of service.
- **Signature**: public void destroy()
- **Clean-Up**: This method is used to release any resources (e.g., database connections, file handles) that the servlet might be holding.
- **Note**: The destroy() method is called only once at the end of the servlet's lifecycle, just before the servlet object is garbage-collected.

👉 Additional Points:

- **Single Instance**: Typically, only one instance of the servlet is created, regardless of the number of requests.
- **Thread Safety**: Since multiple threads may access the service() method simultaneously, developers must ensure that the servlet code is thread-safe.
- **ServletConfig and ServletContext**:
    - **ServletConfig**: Provides configuration information to the servlet.
    - **ServletContext**: Allows the servlet to interact with its environment (e.g., to access application-wide parameters).

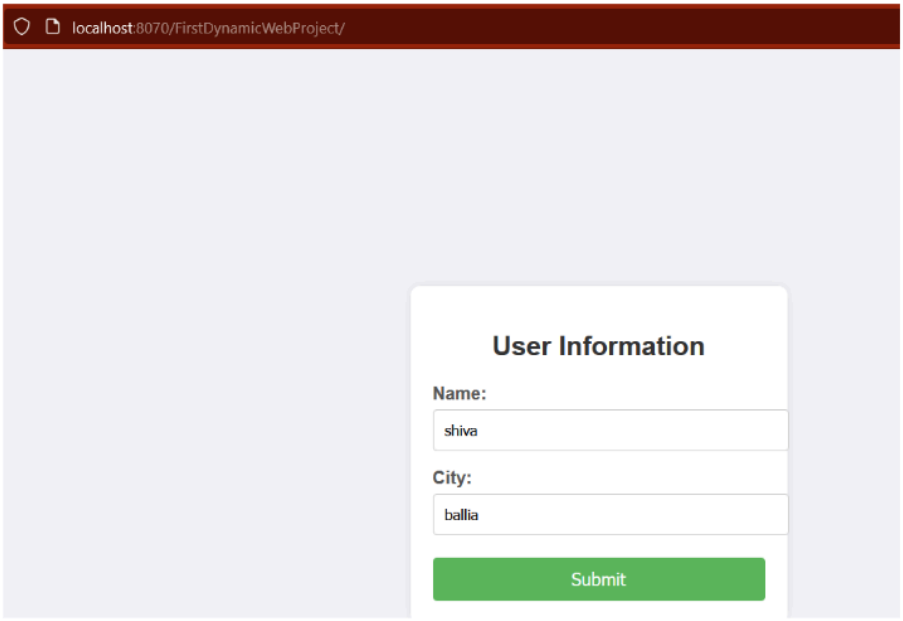
doGet() vs doPost()

In servlets, doGet() and doPost() are two key methods that handle HTTP GET and POST requests, respectively. Understanding the differences and use cases for each is crucial for web application development.

👉 doGet() Method

Register:

http://localhost:8070/FirstDynamicWebProject/



Output:

http://localhost:8070/FirstDynamicWebProject/servletapp?uname=shiva&ucity=ballia


```
Name is : shiva
City is : ballia
```

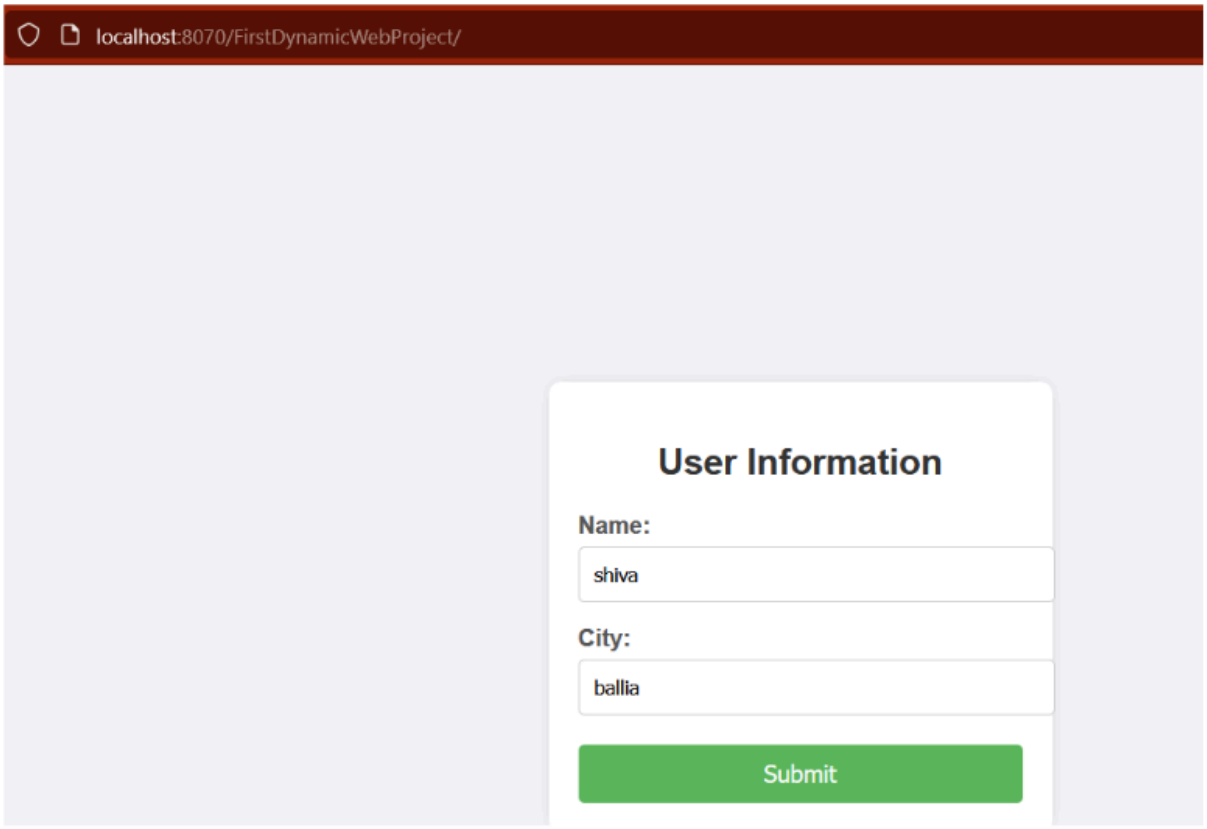
- **Purpose:** Handles HTTP GET requests, typically used to retrieve data from the server.
- **Signature:**

```
protected void doGet(HttpServletRequest req, HttpServletResponse res) throws ServletException, IOException
```

- **Characteristics:**
  - **Request Parameters:** Data is sent via the URL query string (e.g., ?param1=value1&param2=value2).
  - **Data Length:** Limited by the maximum URL length (varies by browser, typically around 2048 characters).
  - **Caching:** Responses to GET requests can be cached by the browser or intermediary servers.
- **Typical Use Cases:**
  - Retrieving data (e.g., displaying a webpage, fetching a list of items).
  - Performing searches based on URL parameters.
  - Navigation actions where no server-side data changes occur.

👉 doPost() Method

http://localhost:8070/FirstDynamicWebProject/



Output:

http://localhost:8070/FirstDynamicWebProject/servletapp

- **Purpose:** Handles HTTP POST requests, generally used to send data to the server for processing (e.g., form submissions).
- **Signature:**

```
protected void doPost(HttpServletRequest req,
```

HttpServletResponse res) throws ServletException, IOException

- **Characteristics:**
  - **Request Parameters:** Data is sent in the request body, not the URL, allowing for larger amounts of data.
  - **Data Length:** Not limited by the URL, suitable for submitting large datasets or files.
  - **Security:** More secure than GET because data is not exposed in the URL (though still needs SSL for proper security).
- **Typical Use Cases:**
  - Submitting form data (e.g., user registration, login forms).
  - Uploading files to the server.
  - Making changes to server data (e.g., creating, updating, or deleting records).

👉 **How doGet() and doPost() method handle in service() method?**

```
public void service(HttpServletRequest req, HttpServletResponse res)
      throws ServletException, IOException {
   String method = req.getMethod();

   if (method.equals("GET")) {
      doGet(req, res);
   } else if (method.equals("POST")) {
      doPost(req, res);
   } else if (method.equals("PUT")) {
      doPut(req, res);
   } else if (method.equals("DELETE")) {
      doDelete(req, res);
   } else {
      // Handle other methods or throw an exception
```

```
        throw new ServletException("Unsupported method: " + method);
    }
}
```

Redirecting Response to JSP or HTML

Redirecting a response in a servlet sends the client to a different resource, such as a JSP or HTML page. This is commonly done using sendRedirect() or RequestDispatcher.

1. Using sendRedirect()

- **Purpose:** Redirects the client to a different URL. This causes the browser to issue a new request to the specified URL.
- **How It Works:**
  - The servlet instructs the client (browser) to request a different URL.
  - A new request is created, so request attributes are not preserved.

```
response.sendRedirect("targetPage.jsp");
```

- **Use Case:** Redirecting to another domain, external website, or when you want the client to initiate a new request.

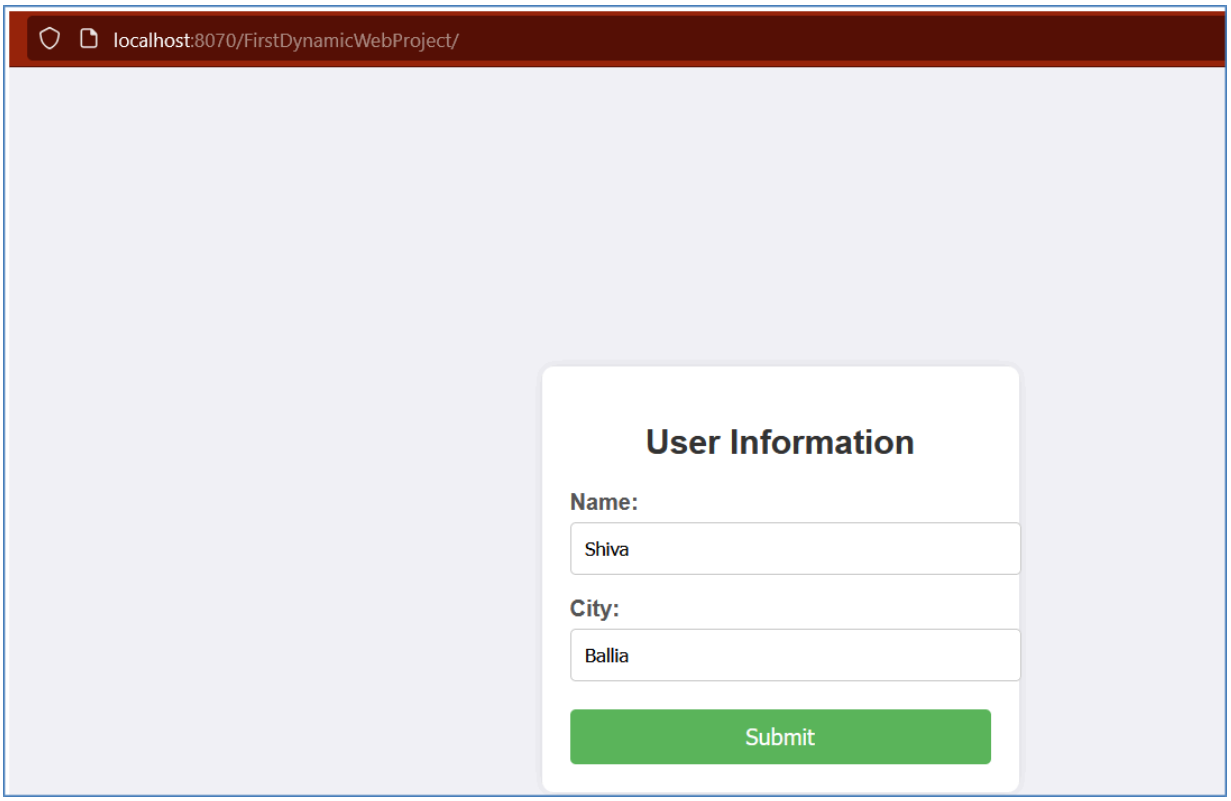
e.g

```
@WebServlet("/firstservlet")
public class FirstServletApp extends HttpServlet {

  public FirstServletApp() {
     System.out.println("Internally constructor called");
  }
```

```
        public void doPost(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException
{

                String name=request.getParameter("uname");
                String city=request.getParameter("ucity");
    response.sendRedirect("/FirstDynamicWebProject/success.jsp");
        }
}
```

Success.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
   pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
<h1> Successful Registration</h1>
</body>
</html>
```



2. Using RequestDispatcher

- **Purpose:** Forwards the request to another resource (JSP or HTML) within the same application.
- **How It Works:**
  - The request is forwarded internally within the server, so the original request and response objects are preserved.
  - No new request is generated; the URL in the browser remains unchanged.

```
RequestDispatcher dispatcher =
request.getRequestDispatcher("targetPage.jsp");
dispatcher.forward(request, response);
```

- **Use Case:** Forwarding to a JSP for further processing or displaying results after servlet processing.

Key Differences:

- **sendRedirect():**
  - Causes a new request.
  - URL in the browser changes.
  - Slower due to the extra client-server round trip.
- **RequestDispatcher.forward():**
  - Keeps the original request and response.
  - URL in the browser does not change.
  - Faster, as it avoids an additional client-server round trip.

Request Dispatching and forward() vs include()

Request dispatching in servlets allows you to forward or include content from one servlet to another. This can be done using the RequestDispatcher interface. The two primary methods provided by RequestDispatcher are forward() and include().

1. RequestDispatcher Interface

- RequestDispatcher is used to dispatch a request from one servlet to another resource (another servlet, JSP, or HTML) on the server.
- **There are two main methods:**
  - **forward(request, response):** Forwards the request from one servlet to another resource on the server.
  - **include(request, response):** Includes the content of another resource (servlet, JSP, HTML) in the response.

2. Forwarding Requests

Important Points:

- **Control Transfer:** The control is completely transferred to another resource (like another servlet) and the client will not know about this change.
- **Single Response:** Only the forwarded servlet's response is sent back to the client. Any content generated before the forward() call is discarded.

forward() in your FirstServletApp:

```
@WebServlet("/firstservlet")
public class FirstServletApp extends HttpServlet {

	public FirstServletApp() {
	System.out.println("Internally constructor called");
	}

	public void doPost(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException
{
	String name = request.getParameter("uname");
	String city = request.getParameter("ucity");
	PrintWriter pw = response.getWriter();

   //This code will not add a response to the output
	pw.println("Before dispatch");

	// Forwarding the request to SecondServletApp
	RequestDispatcher rdsp =
request.getRequestDispatcher("/SecondServletApp");
       rdsp.forward(request, response); // Control is transferred to
SecondServletApp

	// This code will not execute as the control is transferred to
SecondServletApp
	pw.println("Name from 1st : " + name);
	pw.println("City from 1st: " + city);
```

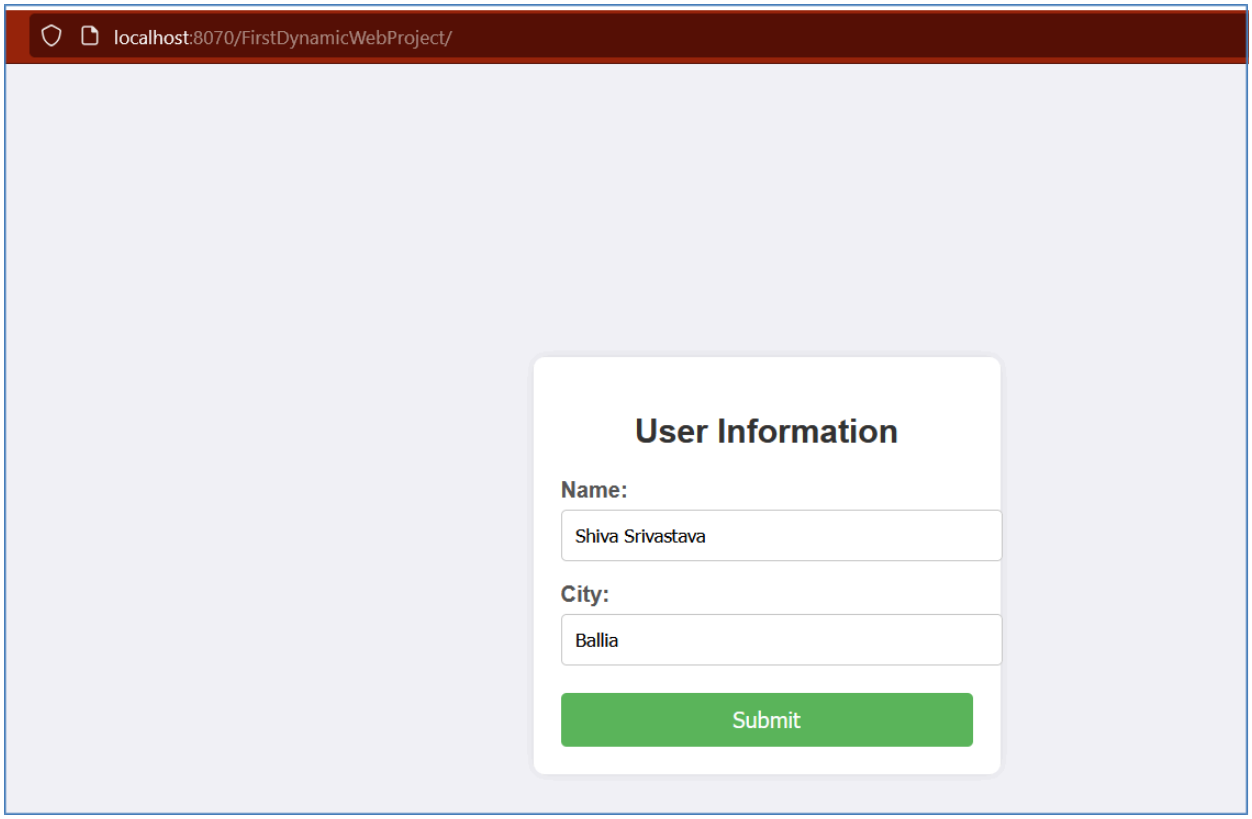
```
    System.out.println("execution come here");
        pw.close();
        }
}
```

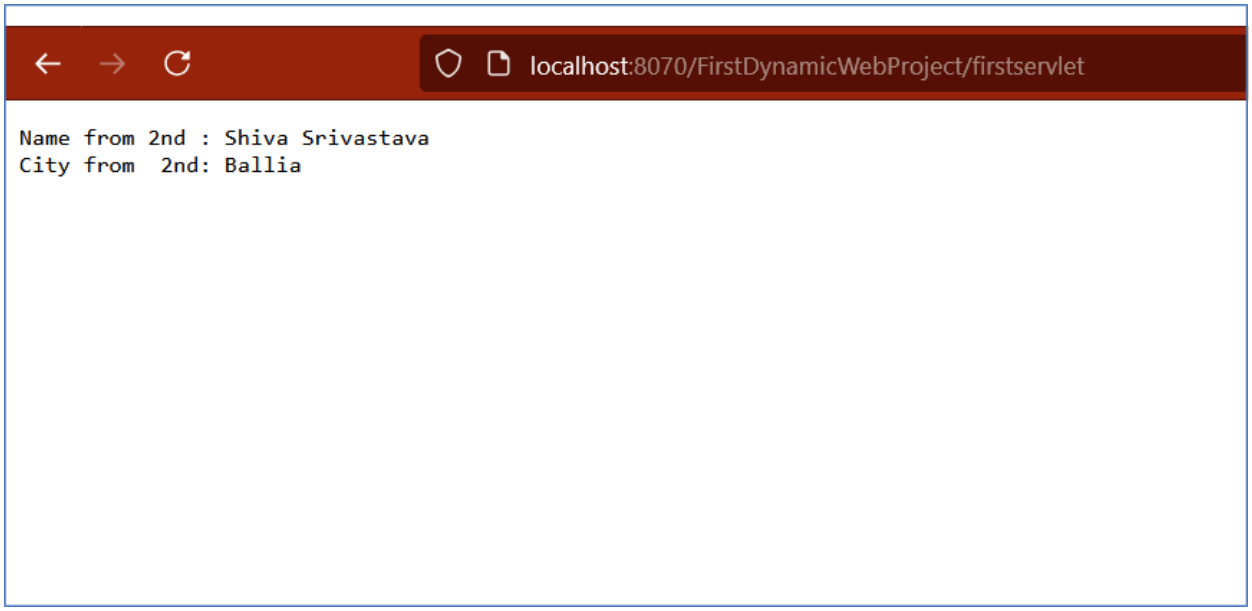
And the SecondServletApp:

```
@WebServlet("/SecondServletApp")
public class SecondServletApp extends HttpServlet {
        private static final long serialVersionUID = 1L;

        public SecondServletApp() {
       System.out.println("SecondServletApp created");
        }

        public void service(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException
{
        String name = request.getParameter("uname");
        String city = request.getParameter("ucity");

        PrintWriter pw = response.getWriter();
        pw.println("Name from 2nd : " + name);
        pw.println("City from 2nd: " + city);

        pw.close();
        }
```

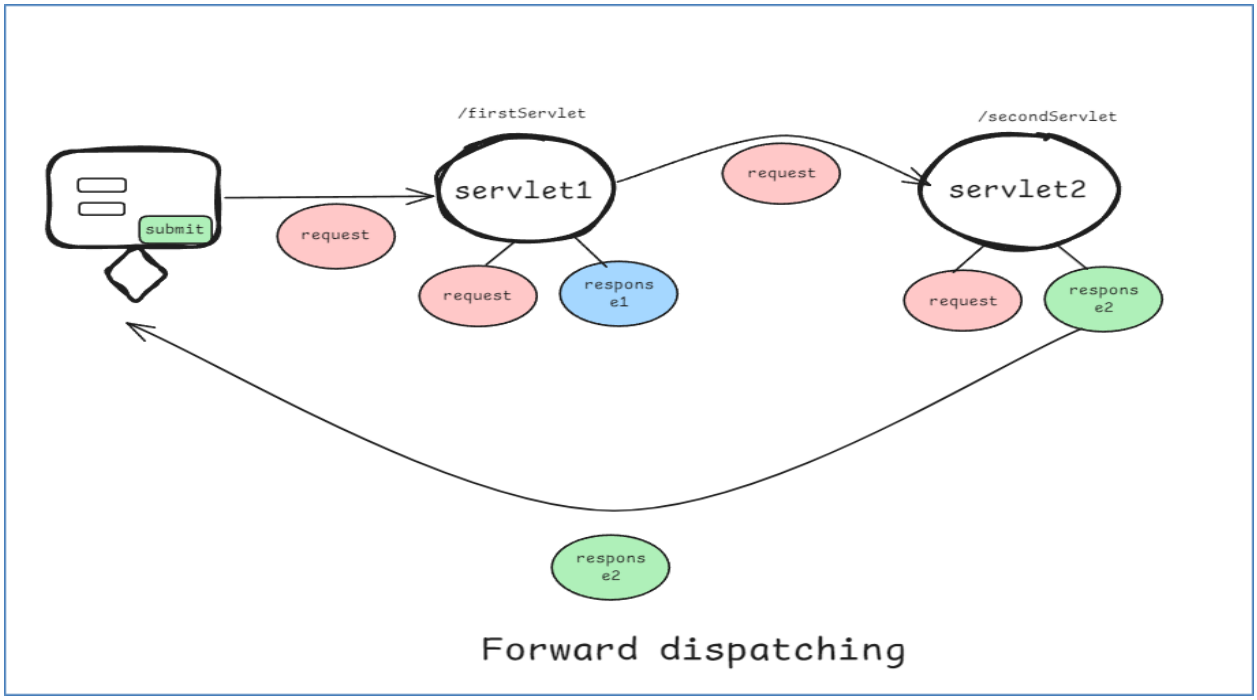
```
}
```

Output:


localhost:8070/FirstDynamicWebProject/
User Information
Name:
Shiva Srivastava
City:
Ballia
Submit


localhost:8070/FirstDynamicWebProject/firstservlet
Name from 2nd : Shiva Srivastava
City from 2nd: Ballia

```
Tomcat v10.1 Server at localhost [Apache Tomcat] C:\Use
Internally constructor called
SecondServletApp created
execution come here
execution come here
```



- The response will only show content from SecondServletApp, as the control is completely transferred there.
- Any content written to the response (using PrintWriter) before the forward() call is discarded and does not appear in the final output. This is because forward() transfers control to another resource, and the response buffer is cleared before the forwarding occurs.

3. Including Requests

Important Points:

- **Control Retention:** The control returns back to the original servlet after including the content of another resource.
- **Combined Response:** The response from the included resource is combined with the response of the original servlet.

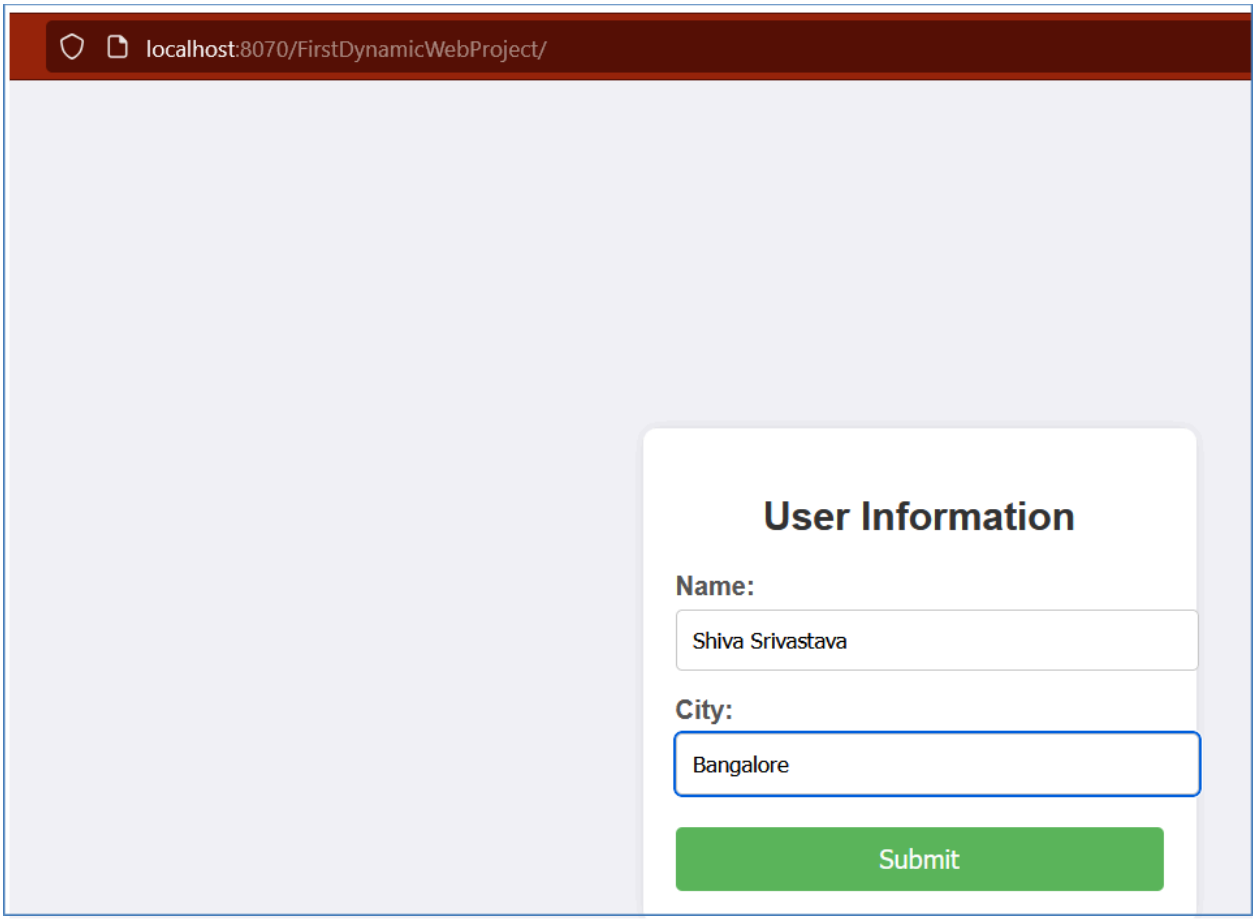
FirstServletApp to use include():

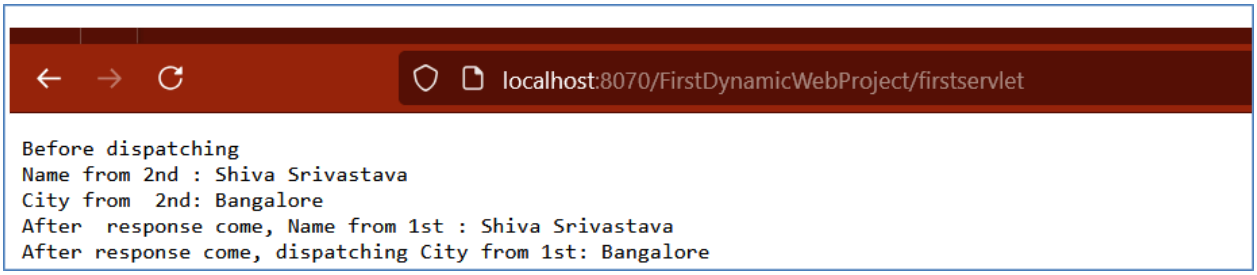
```
@WebServlet("/firstservlet")
public class FirstServletApp extends HttpServlet {

	public FirstServletApp() {
 System.out.println("Internally constructor called");
	}

	public void doPost(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException
{
		String name = request.getParameter("uname");
		String city = request.getParameter("ucity");
		PrintWriter pw = response.getWriter();

		pw.println("Before dispatch");

		// Including the response from SecondServletApp
		RequestDispatcher rdsp =
request.getRequestDispatcher("/SecondServletApp");
		rdsp.include(request, response); // Control is temporarily
```

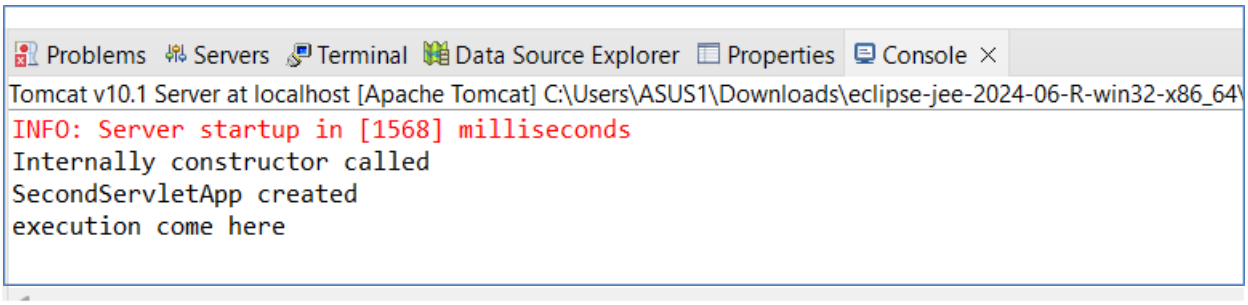
```
transferred to SecondServletApp and returns back

// This code will execute after including the SecondServletApp response
        pw.println("Name from 1st : " + name);
        pw.println("City from 1st: " + city);

  System.out.println("execution come here");
        pw.close();
        }
}
```

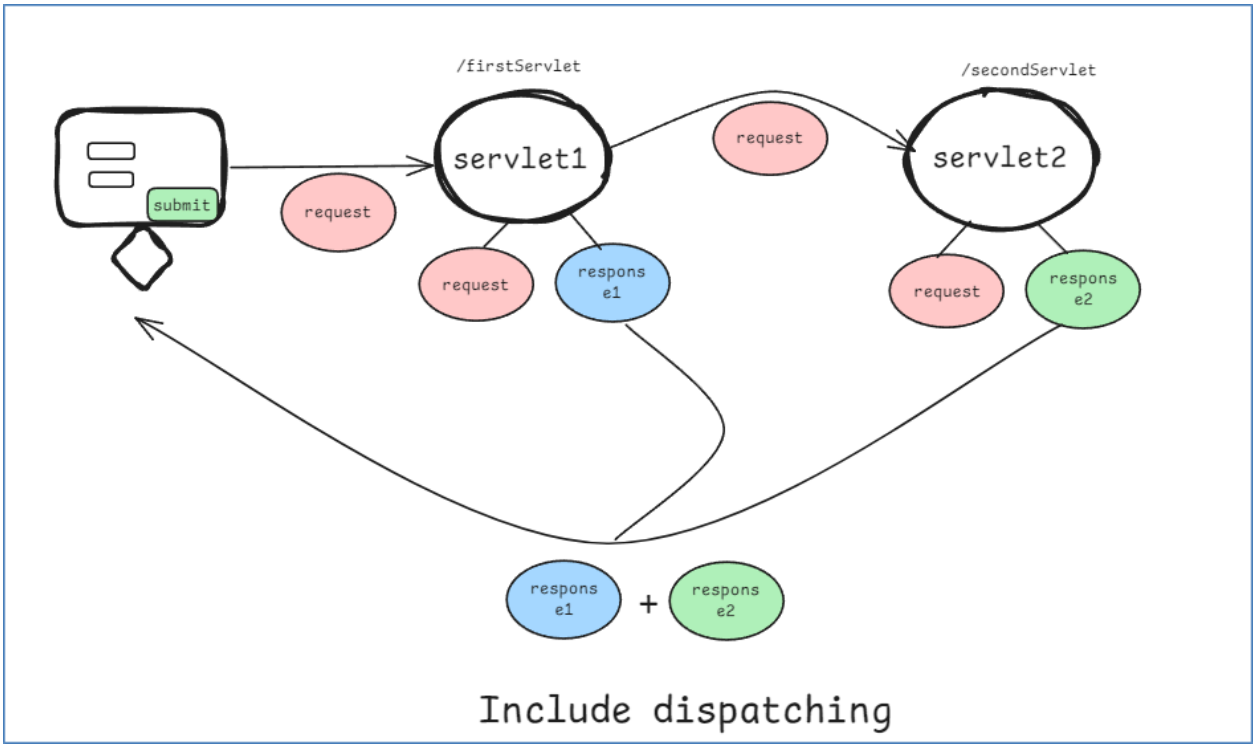
And SecondServletApp remains the same as before.

Output:


localhost:8070/FirstDynamicWebProject/
User Information
Name:
Shiva Srivastava
City:
Bangalore
Submit


localhost:8070/FirstDynamicWebProject/firstservlet
Before dispatching
Name from 2nd : Shiva Srivastava
City from 2nd: Bangalore
After response come, Name from 1st : Shiva Srivastava
After response come, dispatching City from 1st: Bangalore


Problems
Servers
Terminal
Data Source Explorer
Properties
Console
Tomcat v10.1 Server at localhost [Apache Tomcat] C:\Users\ASUS1\Downloads\eclipse-jee-2024-06-R-win32-x86_64\
INFO: Server startup in [1568] milliseconds
Internally constructor called
SecondServletApp created
execution come here



- The response will include content from both FirstServletApp and SecondServletApp, as the control returns to FirstServletApp after include().
- **Before include()**: Any content written to the response prior to calling include() will be retained and included in the final output along with the included servlet's content.
- **After include()**: Code after the include() method will continue to execute, and its output will also be added to the response, resulting in a combined response from both the original servlet and the included servlet.

👉 Summary:

- **forward():**
  - Transfers control to another servlet.
  - Response is generated by the forwarded servlet only.

    - No further response content is added after forward().
- **include():**
    - Temporarily transfers control to another servlet.
    - Combined response from both servlets is returned to the client.
    - The control returns to the original servlet after the included servlet completes its task.

This way, forward() is used when you want another resource to completely handle the request, and include() is used when you want to include additional content from another resource in your response.

HttpSession

👉 Session Creation & Management:

- The HttpSession object allows you to store user-specific data (like name and city) across multiple requests.
- **Creating/Accessing Session:** Use request.getSession() to create a new session if one does not exist or to access the current session.
- **Session Attributes:** You can store attributes using session.setAttribute("key", value) and retrieve them with session.getAttribute("key").

👉Redirecting Between Servlets:

- **Storing Data in Session:** Data stored in one servlet can be accessed in another servlet by the same user session. This is useful when redirecting between different parts of the application.
- **Redirecting:** Use response.sendRedirect("URL") to redirect the user to another servlet while keeping the session data intact.

👉Accessing Session Data:

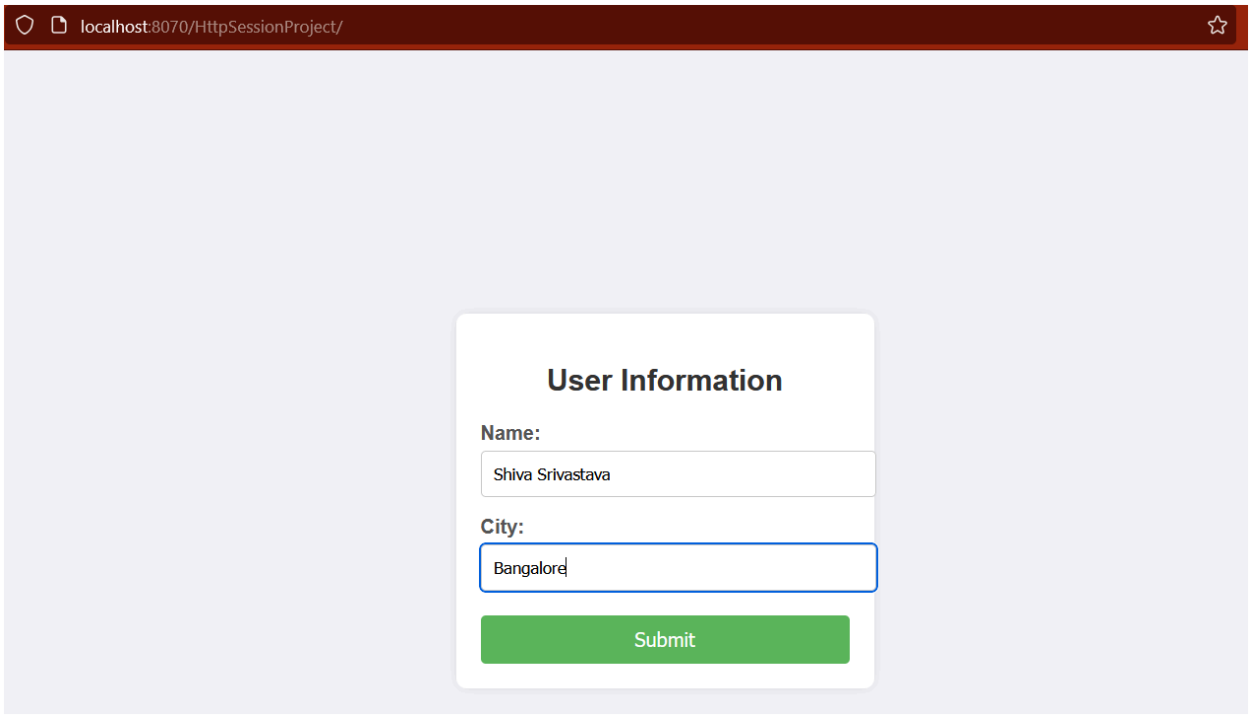
- **Retrieving Existing Session:** Use request.getSession(false) in the second servlet to retrieve the current session without creating a new one if it doesn't exist.
- **Accessing Attributes:** Retrieve the stored attributes using session.getAttribute("key") and use them as needed.

```java
@WebServlet("/FirstServletApp")
public class FirstServletApp extends HttpServlet {
	public void service(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException
{
	String name = request.getParameter("uname");
	String city = request.getParameter("ucity");

	// Creating or accessing the session
	HttpSession session = request.getSession();
	session.setAttribute("name", name);  // Storing data in session
	session.setAttribute("city", city);

	// Redirecting to another servlet
	response.forward ("SecondServletApp");
	}
}
```

```java
@WebServlet("/SecondServletApp")
public class SecondServletApp extends HttpServlet {
	public void service(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException
{
	// Accessing the current session without creating a new one
	HttpSession session = request.getSession(false);

	String name = (String) session.getAttribute("name");  //
Retrieving session data
```

```
        String city = (String) session.getAttribute("city");

        // Displaying the session data
        PrintWriter pw = response.getWriter();
        pw.println("<h1>Name: " + name + "</h1>");
        pw.println("<h1>City: " + city + "</h1>");
        pw.close();
        }
}
```

Output:



Name is Shiva Srivastava

City is Bangalore

👉 Summary:

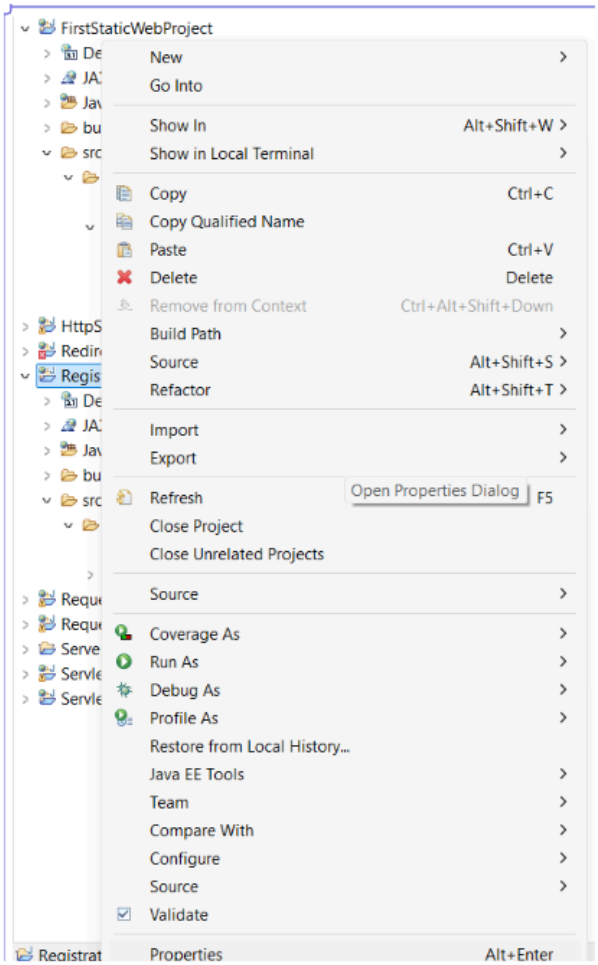
- Use HttpSession to manage user data across multiple servlets.
- Store data with session.setAttribute(), retrieve it using session.getAttribute(), and share it across servlets even after redirection.
- Always ensure the session is retrieved correctly (getSession(false)) when accessing it in subsequent servlets to avoid creating a new session unnecessarily.

Registration App using Servlet and JDBC

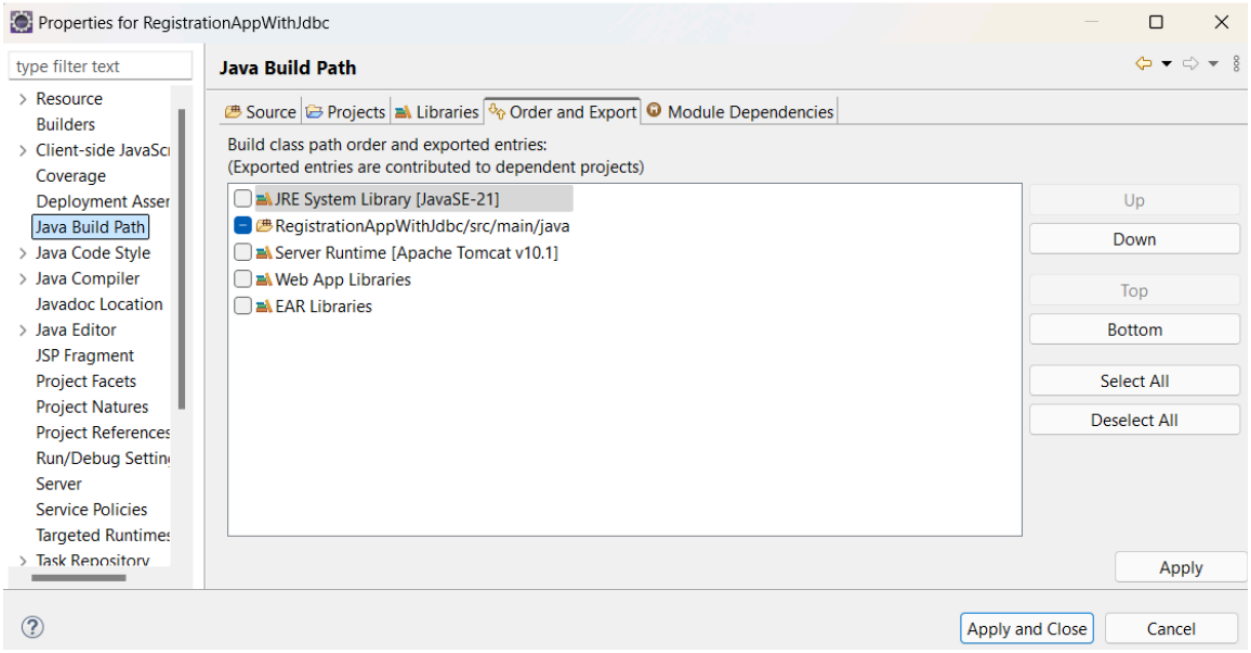
👉 Set up the Eclipse environment for this project.

Step 1: Create a Dynamic Project

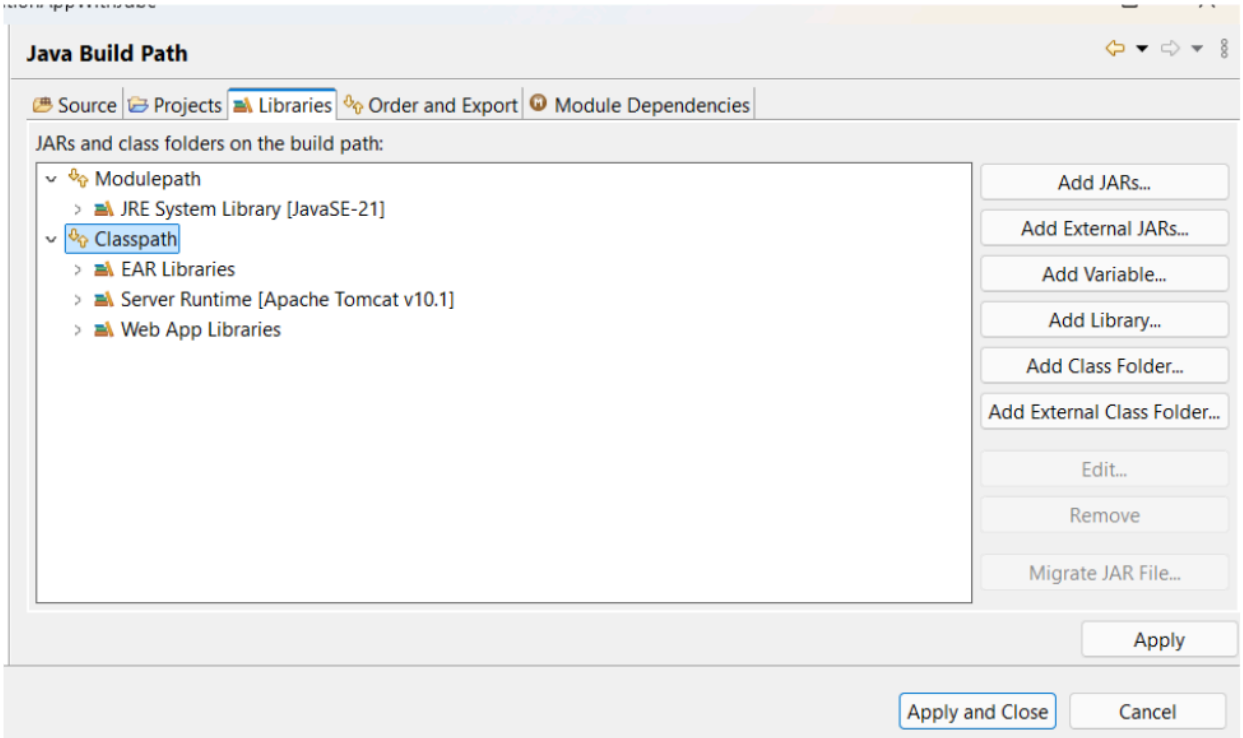
Step 2: Go to properties



Step 3: Go to Java Build Path.



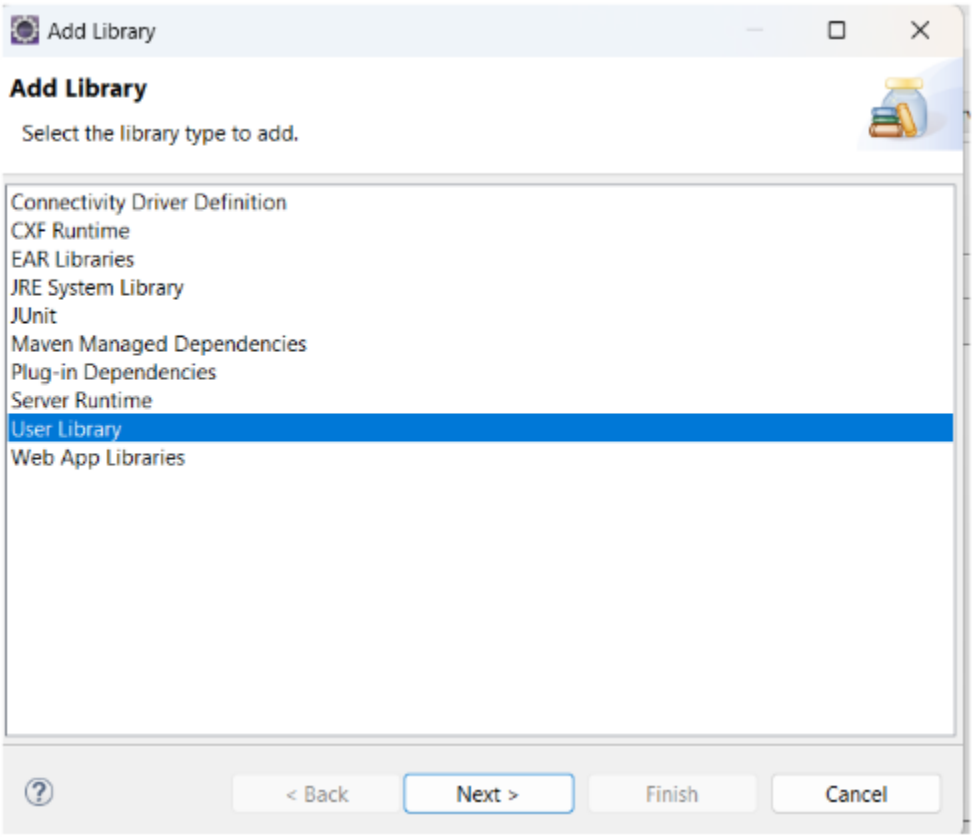
Step 4: Go to libraries



=> Click on Classpath

=> Add library

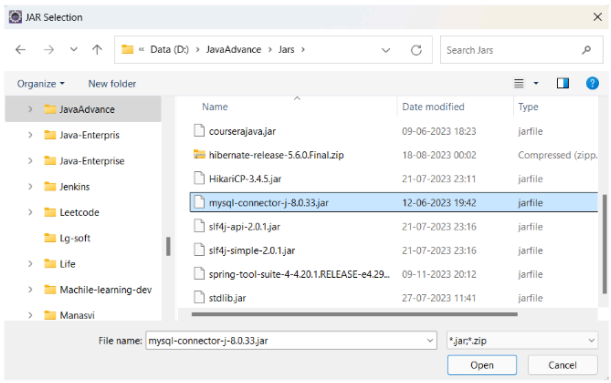
Step5: Select User library

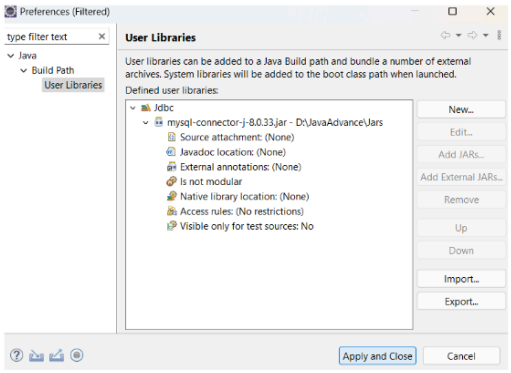


=> Click Next

Step 6: Create a new library with the name Jdbc.

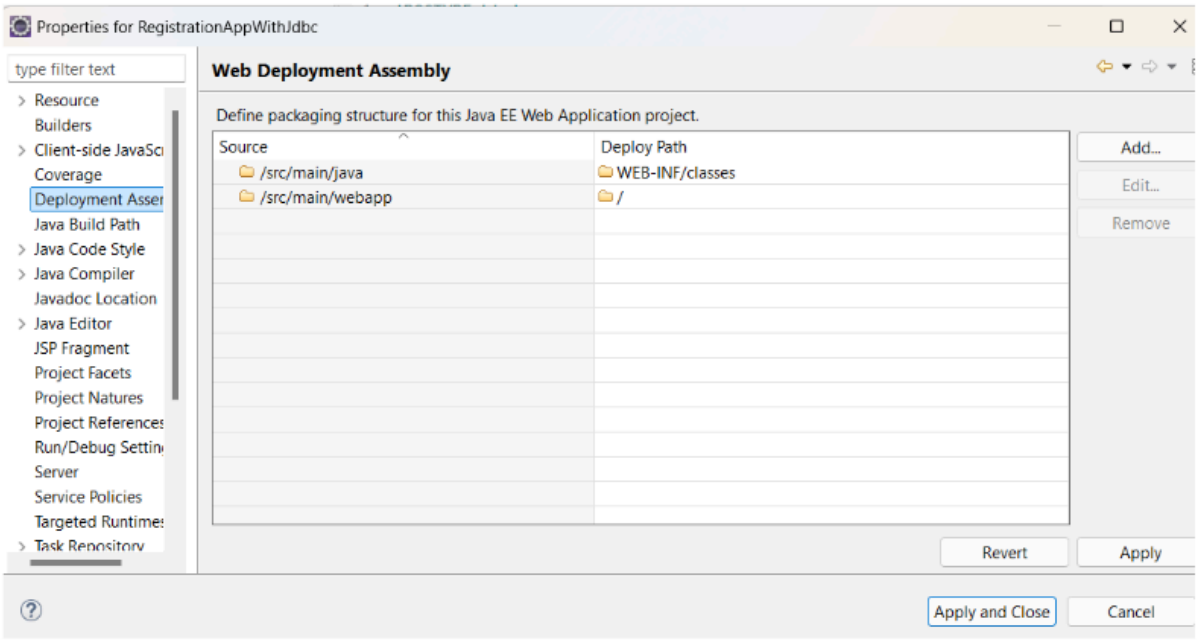
Step 7: Add the external JAR inside the Jdbc folder.





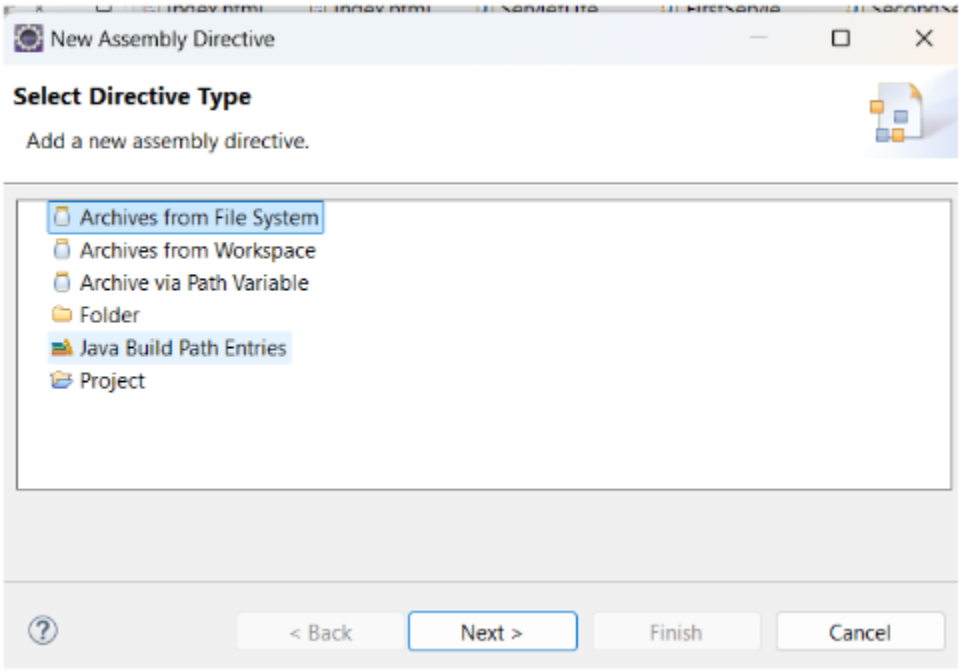
Apply and Close

Step 8: Go to Deployment Assembly



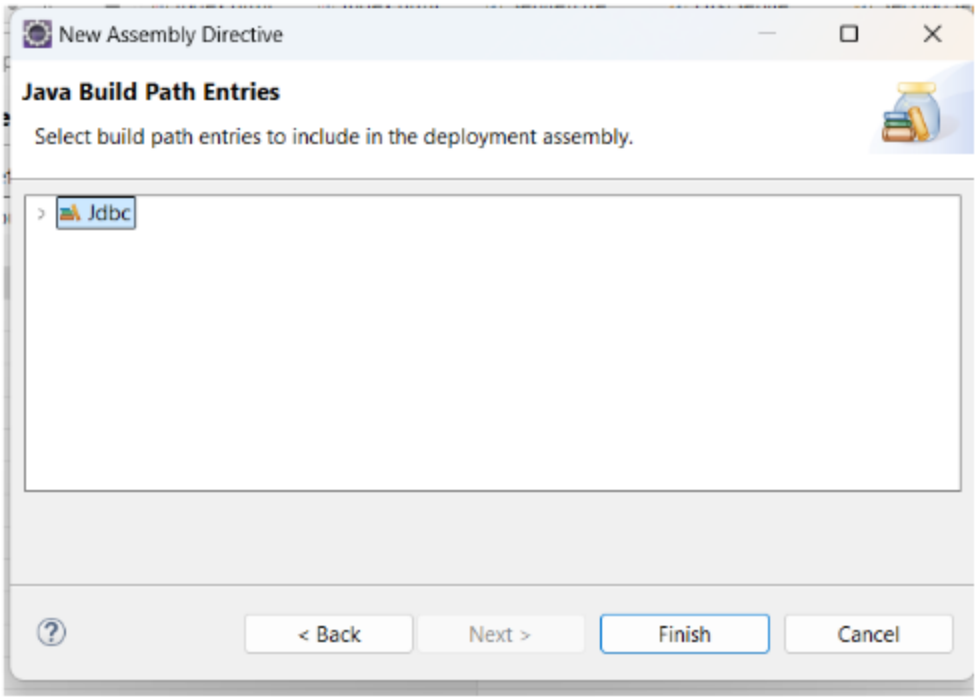
Select / folder and then click on Add

Step9: Select Java Build Path Entries



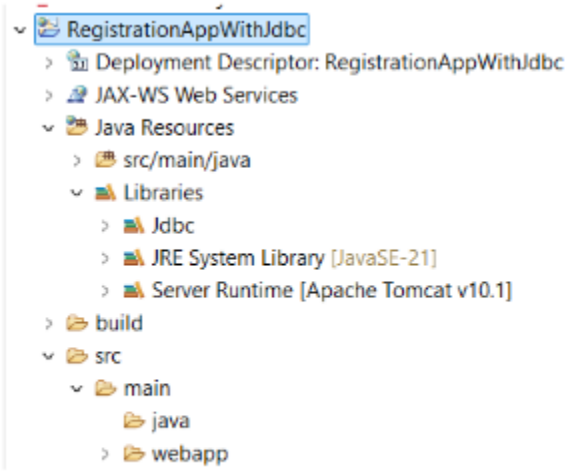
Click Next

Step 10: Select Jdbc folder.



Click finish

Project Structure:



👉**Code Flow**

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>User Registration Form</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f4;
      margin: 0;
      padding: 0;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
    }
    form {
      background-color: #fff;
      padding: 20px;
      border-radius: 8px;
      box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
      width: 300px;
    }
    h2 {
```

```
        margin-bottom: 20px;
        font-size: 24px;
        text-align: center;
        color: #333;
    }
    label {
        margin-bottom: 10px;
        display: block;
        font-weight: bold;
        color: #555;
    }
    input[type="text"],
    input[type="email"],
    input[type="password"] {
        width: 100%;
        padding: 8px;
        margin-bottom: 15px;
        border: 1px solid #ccc;
        border-radius: 4px;
        box-sizing: border-box;
    }
    input[type="submit"] {
        width: 100%;
        padding: 10px;
        background-color: #28a745;
        border: none;
        border-radius: 4px;
        color: #fff;
        font-size: 16px;
        cursor: pointer;
```

```
        }
        input[type="submit"]:hover {
            background-color: #218838;
        }
    </style>
</head>
<body>
    <form action="./RegisterServlet" method="POST">
        <h2>User Registration</h2>
        <label for="uname">Name:</label>
        <input type="text" id="uname" name="uname" maxlength="255"
required>
        <label for="email">Email:</label>
        <input type="email" id="email" name="uemail" maxlength="255"
required>
        <label for="upassword">Password:</label>
        <input type="password" id="upassword" name="upassword"
maxlength="255" required>
        <label for="ucity">City:</label>
        <input type="text" id="ucity" name="ucity" maxlength="255">
        <input type="submit" value="Register">
    </form>
</body>
</html>
```

RegisterServlet.java

```
import java.io.IOException;
```

```java
import java.io.PrintWriter;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
@WebServlet("/RegisterServlet")
public class RegisterServlet extends HttpServlet {

	public void service(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException
{

		String name=request.getParameter("uname");
		String email=request.getParameter("uemail");
		String password=request.getParameter("upassword");
		String city=request.getParameter("ucity");


		try {
			//loading and register driver
			Class.forName("com.mysql.cj.jdbc.Driver");

			String url="jdbc:mysql://localhost:3306/telusko_db";
			String username="root";
			String passwd="root";
```

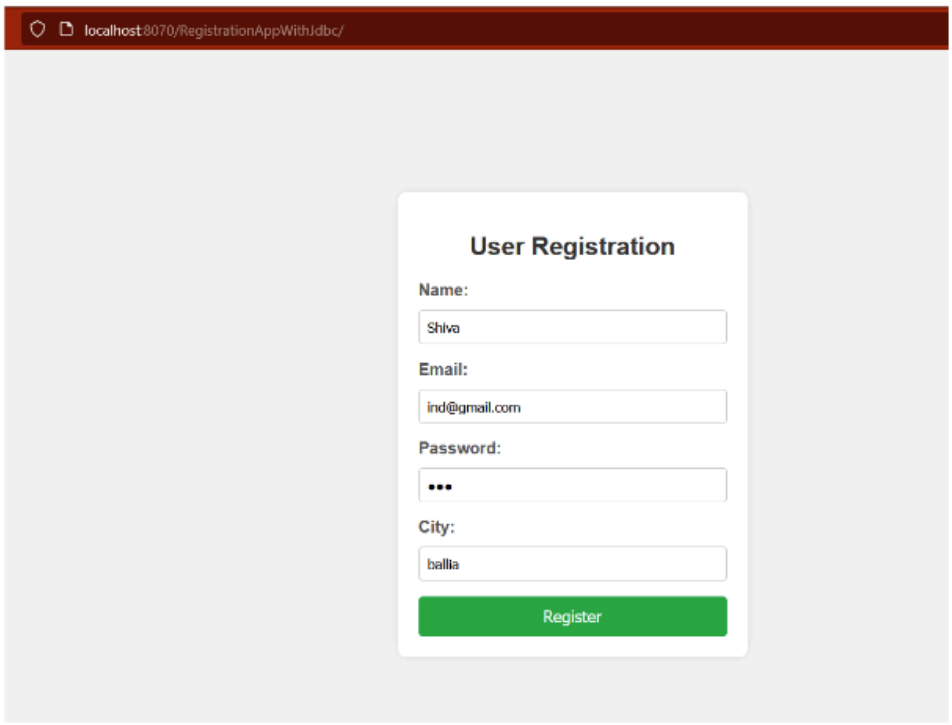
```
                    //create connection
                    Connection
connt=DriverManager.getConnection(url,username,passwd);

                    String query = "INSERT INTO PERSONINFO
(UNAME, EMAIL,UPASSWORD, UCITY) VALUE(?,?,?,?)";
                    PreparedStatement
pstmt=connt.prepareStatement(query);

                    pstmt.setString(1, name);
                    pstmt.setString(2, email);
                    pstmt.setString(3, password);
                    pstmt.setString(4, city);

                    int rowAff=pstmt.executeUpdate();
                    PrintWriter pw=response.getWriter();
                    if(rowAff!=0) {
                            pw.println("<h1> Data inserted Succesfully");
                    }else {
                     pw.println("<h1> Data not inserted</h1>");
                    }
                    pw.close();

            } catch (ClassNotFoundException | SQLException e) {

                    e.printStackTrace();
            }

    }
}
```

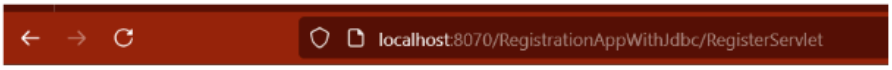
👉Before running the program:

1. Create database with name ***telusko_db***
2. Create table with name ***personinfo***

```
mysql> create database telusko_db;
Query OK, 1 row affected (0.03 sec)

mysql> use telusko_db;
Database changed
mysql> create table personinfo (uname varchar(255), email varchar(255), upassword varchar(255), ucity varchar(255));
Query OK, 0 rows affected (0.08 sec)
```

Output:





Data inserted Succesfully

```
mysql> select * from personinfo;
+-------+---------------+-----------+--------+
| uname | email         | upassword | ucity  |
+-------+---------------+-----------+--------+
| Shiva | ind@gmail.com | 123       | ballia |
+-------+---------------+-----------+--------+
1 row in set (0.01 sec)
```

Introduction to JSP

1. Why Do We Need JSP?

- **Dynamic Web Content:** JSP (JavaServer Pages) allows developers to create dynamic web content by embedding Java code directly within HTML pages. This is useful for generating dynamic pages like displaying user data, handling forms, and interacting with databases.
- **Separation of Concerns:** JSP helps separate the presentation layer (HTML/CSS) from the business logic (Java code). This allows web designers and developers to work together more efficiently, with designers focusing on the layout and developers on the functionality.
- **Ease of Development:** JSP simplifies the creation of web pages by allowing developers to use Java classes and methods directly within HTML tags. This reduces the need for complex servlet code to generate HTML output.

2. JSP Lifecycle

JSP has a well-defined lifecycle that consists of several stages from creation to destruction. Understanding this lifecycle helps in optimizing and managing resources in a web application.

1. **Translation Phase**:
    - **JSP to Servlet Conversion:** When a JSP page is requested for the first time, the JSP engine (e.g., Jasper in Tomcat) converts the JSP file into a corresponding Java Servlet class. This step involves translating JSP syntax (such as scriptlets, expressions, and directives) into valid Java code.

    - **Example:** <%= expression %> in JSP is converted to out.print(expression); in the servlet.
2. **Compilation Phase**:
    - **Servlet Compilation:** The generated servlet class is then compiled into bytecode, creating a .class file. This compiled servlet can now be executed by the servlet container.
3. **Loading & Initialization**:
    - **Class Loading:** The servlet class is loaded into the JVM (Java Virtual Machine) by the servlet container.
    - **Initialization (jspInit() method):** The jspInit() method is called once when the servlet is first loaded. It's equivalent to the init() method in a regular servlet, used to initialize resources.
4. **Request Processing**:
    - **Service Method (_jspService()):** Each time a request is made to the JSP, the _jspService() method is invoked. This method processes the request, generates the response, and sends it back to the client. This is where most of the JSP-generated code (including embedded Java) is executed.
5. **Destruction**:
    - **Destroy (jspDestroy() method):** When the JSP is no longer needed, the jspDestroy() method is called to clean up any resources (like closing database connections) before the servlet is removed from memory.

3. How JSP is Converted to a Servlet and Rendered

- **JSP to Servlet Conversion:**
    - The JSP file (with .jsp extension) is parsed and converted into a Java servlet by the JSP engine (like Jasper in Tomcat). This servlet is essentially a Java class that extends

HttpServlet and contains the logic to generate the HTML content dynamically.

- **Servlet Compilation and Execution:**
  - After conversion, the servlet is compiled into bytecode, loaded into memory, and executed just like a regular servlet. The output of the servlet (typically HTML) is sent as a response to the client's browser.
- **Rendering the View:**
  - The servlet generates the HTML content dynamically based on the logic defined in the JSP, which is then rendered by the client's browser as a web page.

4. Tomcat and Jasper in JSP Lifecycle

- **Tomcat:**
  - Apache Tomcat is a popular open-source servlet container that serves as the runtime environment for executing Java servlets and JSPs. It handles the entire lifecycle of JSPs, from translation to servlet execution.
- **Jasper:**
  - Jasper is the JSP engine used by Tomcat. It is responsible for translating JSP files into servlets and managing their compilation. Jasper handles the conversion of JSP syntax into valid Java code, ensuring that the JSP page can be executed as a servlet.

👉**Conclusion:**

JSP simplifies web development by allowing Java code to be embedded in HTML, separating the presentation layer from the business logic. Understanding the JSP lifecycle, from translation to servlet execution, is crucial for optimizing web applications. Tools like Tomcat and Jasper play a key role in managing this lifecycle, making JSP a powerful tool for dynamic web content generation.

JSP Tags and Web App using JSP

JSP (JavaServer Pages) provides a powerful and flexible way to develop dynamic web applications by allowing Java code to be embedded directly in HTML. JSP tags and implicit objects play a crucial role in this process.

1. JSP Tags Overview

JSP tags are used to insert Java code, manage session and request data, and import classes into a JSP page. Below are the main types of JSP tags:

1. **Scriptlet Tag (<% ... %>)**
    - **Purpose:** Allows embedding of Java code directly within the HTML page. The code inside the scriptlet is executed each time the JSP page is requested.
    - Scriptlet tags are used to embed Java code directly within a JSP page. The code inside a scriptlet is executed each time the JSP page is requested, and it gets embedded within the generated servlet's _jspService() method.
    - **Example:**

```
<%
int age = 22;
String name = "Shiva";
System.out.println("Name is " + name + " and age is " + age);
%>
```

    - **Use Case:** Perform server-side computations, manage conditions, or call Java methods.

2. **Declaration Tag (<%! ... %>)**
    - **Purpose:** Declares methods, variables, or classes at the servlet class level. This code is placed outside the _jspService() method, meaning it's not executed with every request.
    - **Example:**

```
<%!
private int x = 23;
private String secondName = "Harsh";
%>
```

    - **Use Case:** Define variables or methods that are needed throughout the lifecycle of the JSP servlet.
3. **Expression Tag (<%= ... %>)**
    - **Purpose:** Outputs the result of a Java expression directly to the client. The expression is evaluated, and its value is converted to a string and inserted into the HTML.

```
<h1>My Second name is <%= secondName %></h1>
```

    - **Use Case:** Display calculated or dynamic values directly in the HTML output.
4. **Directive Tag (<%@ ... %>)**
    - **Purpose:** Provides instructions to the JSP engine regarding the overall structure of the servlet. Common directives include page settings, importing Java classes, and managing error handling.
    - **Types:**
        - **Page Directive:** Configures page-level settings like importing classes, setting content types, etc.

```
<%@ page import="java.util.Date" %>
```

- **Include Directive:** Includes a static file at the time of JSP translation.

```
<%@ include file="header.jsp" %>
```

- **Taglib Directive:** Declares a custom tag library for use in the JSP.

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

2. Implicit Objects in JSP

JSP provides several implicit objects that are automatically available within a JSP page. These objects represent various components of the request and response cycle.

1. **request**
   - **Type:** HttpServletRequest
   - **Purpose:** Represents the HTTP request data sent by the client, including form data, request headers, etc.

```
String str1 = request.getParameter("uname");
```

2. **response**
   - **Type:** HttpServletResponse
   - **Purpose:** Represents the HTTP response returned to the client, allowing you to set response headers, status codes, and send data back.
   - **Example:**

```
response.setContentType("text/html");
```

3. **session**

- **Type:** HttpSession
- **Purpose:** Provides a way to store data specific to a user across multiple requests (sessions).
- **Example:**

```
HttpSession session = request.getSession();
session.setAttribute("degree", "B.Tech CSE");
```

4. **out**
    - **Type:** JspWriter
    - **Purpose:** Used to send output to the client. It's essentially the JSP equivalent of PrintWriter in servlets.
    - **Example:**

```
out.println("<h1>Hello, World!</h1>");
```

5. **application**
    - **Type:** ServletContext
    - **Purpose:** Represents the web application context, shared across all users and requests. Useful for storing application-wide data.
    - **Example:**

```
application.setAttribute("appName", "My Web App");
```

6. **config**
    - **Type:** ServletConfig
    - **Purpose:** Provides configuration information for the JSP's servlet, such as initialization parameters.
    - **Example:**

```
String servletName = config.getServletName();
```

7. **pageContext**

- **Type:** PageContext
- **Purpose:** Provides access to all the other implicit objects and also manages the lifecycle of JSP-specific resources.
- **Example:**

```
pageContext.setAttribute("name", "Shiva");
```

8. **page**
   - **Type:** Object
   - **Purpose:** Refers to the current JSP page. It's similar to this in a Java class.
   - **Example:**

Note: Generally not used directly, but available if needed

9. **exception**
   - **Type:** Throwable
   - **Purpose:** Used in JSP error pages to handle exceptions that occur during the processing of the JSP.
   - **Example:**

```
<h1>Error: <%= exception.getMessage() %></h1>
```

3. JSP and Servlets Integration

- **JSP as a Servlet:** Behind the scenes, JSPs are converted to servlets by the JSP engine (like Jasper in Tomcat). This means that all the JSP code eventually gets translated into Java code inside a servlet's _jspService() method.
- **Tomcat and Jasper:** Tomcat is a popular servlet container that executes servlets and JSPs. Jasper is Tomcat's JSP engine, responsible for converting JSP files into servlets, compiling them, and managing their execution.

4.Life Cycle of JSP Imlicit Object:

1. request Object (HttpServletRequest)

- Creation: Created at the beginning of each client request.
- Scope: Lasts for the duration of a single request.
- Destruction: Destroyed after the response is sent.

2. session Object (HttpSession)

- Creation: Created when a client accesses the application for the first time.
- Scope: Lasts across multiple requests from the same client.
- Destruction: Destroyed when the session times out or is invalidated.

3. application Object (ServletContext)

- Creation: Created when the web application starts.
- Scope: Shared across the entire application.
- Destruction: Destroyed when the application is stopped or undeployed.

4. pageContext Object (PageContext)

- Creation: Created for each JSP page request.
- Scope: Limited to the duration of the request within the JSP page.
- Destruction: Destroyed after the JSP page finishes processing.

5. page Object (Object)

- Creation: Refers to the current JSP page instance.
- Scope: Limited to the JSP page itself.

- Destruction: Destroyed when the JSP page is unloaded or the request ends.

👉 **Code Flow:**

index.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body style="font-family: Arial, sans-serif; background-color: #f4f4f9; display: flex; justify-content: center; align-items: center; height: 100vh; margin: 0;">
  <form style="background-color: #ffffff; padding: 20px; border-radius: 8px; box-shadow: 0 0 10px rgba(0, 0, 0, 0.1); width: 300px;"  action="tag.jsp" method="post">
      <h2 style="text-align: center; color: #333333;">User Information</h2>

      <div style="margin-bottom: 15px;">
          <label for="uname" style="display: block; font-weight: bold; margin-bottom: 5px; color: #555555;">Name:</label>
          <input type="text" id="uname" name="uname" style="width: 100%; padding: 10px; border: 1px solid #cccccc; border-radius: 4px;">
      </div>
      <div style="margin-bottom: 20px;">
          <label for="ucity" style="display: block; font-weight: bold; margin-bottom: 5px; color: #555555;">City:</label>
```

```
        <input type="text" id="ucity" name="ucity" style="width:
100%; padding: 10px; border: 1px solid #cccccc; border-radius: 4px;">
      </div>
      <input type="submit" value="Submit" style="width: 100%;
padding: 10px; background-color: #5cb85c; color: white; border: none;
border-radius: 4px; font-size: 16px; cursor: pointer;">
  </form>
</body>
</html>
```

tag.jsp

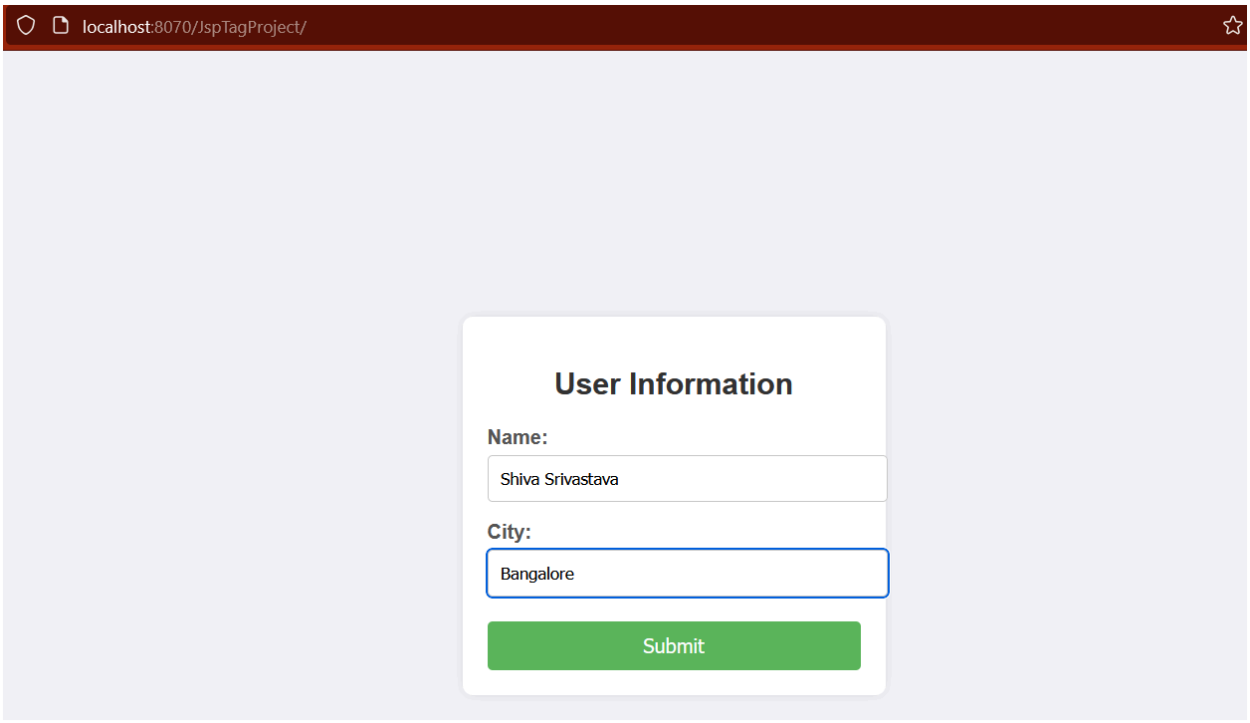
```
<%@ page language="java" contentType="text/html; charset=UTF-8"
  pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>JSP Example</title>
</head>
<body>
<!-- Scriptlet tag  -->
<%
int age = 22;
String name = "Shiva";
System.out.println("Name is " + name + " and age is " + age);
%>
<!-- Declaration tag -->
<%!
```

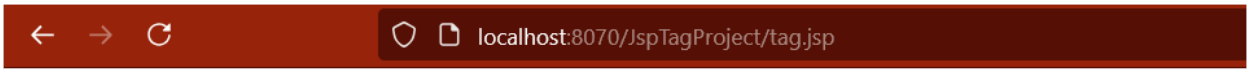
```
private int x = 23;
private String secondName = "Harsh";
%>
<!-- Expression tag -->
<h1>My Second name is <%= secondName %></h1>
<!-- Directive tag -->
<%@ page import="java.util.Date" %>
<%
Date date = new Date();
%>
<h2>Today's date is <%= date %></h2>
<!-- Implicit object -->
<!-- Session object -->
<%
out.println("<h2>.................Session Object.............</h2>");
String degree = (String) session.getAttribute("degree");
String gender = (String) session.getAttribute("gender");  // Corrected
method
out.println("<h2>My degree is " + degree + " and gender is " + gender
+ "</h2>");
%>
<!-- Request object and PrintStream object (out) -->
<%
out.println("<h2>.............Request Object.................</h2>");
String str1 = request.getParameter("uname");
String str2 = request.getParameter("ucity");
%>
<h2>My name is <%= str1 %> and city is <%= str2 %></h2>
</body>
</html>
```

MyServlet.java

```
import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
/**
* Servlet implementation class MyServlet
*/
@WebServlet("/MyServlet")
public class MyServlet extends HttpServlet {
	private static final long serialVersionUID = 1L;
	public void service(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException
{
		HttpSession session=	request.getSession();
		session.setAttribute("degree", "B.Tech CSE");
		session.setAttribute("gender", "Male");
	}
}
```

Output:


localhost:8070/JspTagProject/
User Information
Name:
Shiva Srivastava
City:
Bangalore
Submit



My Second name is Harsh

Today's date is Sat Aug 17 12:32:45 IST 2024

.....Session Object.....

My degree is null and gender is null

.....Request Object.....

My name is Shiva Srivastava and city is Bangalore

👉Conclusion

JSP offers a powerful way to build dynamic web applications by combining the ease of HTML with the power of Java. By using various JSP tags and implicit objects, developers can efficiently manage the web application's presentation layer while keeping the logic encapsulated in servlets and JavaBeans. Understanding the lifecycle of JSPs and their integration with servlets is crucial for optimizing and managing web applications effectively.

Servlet vs JSP

👉JavaServer Pages (JSP):

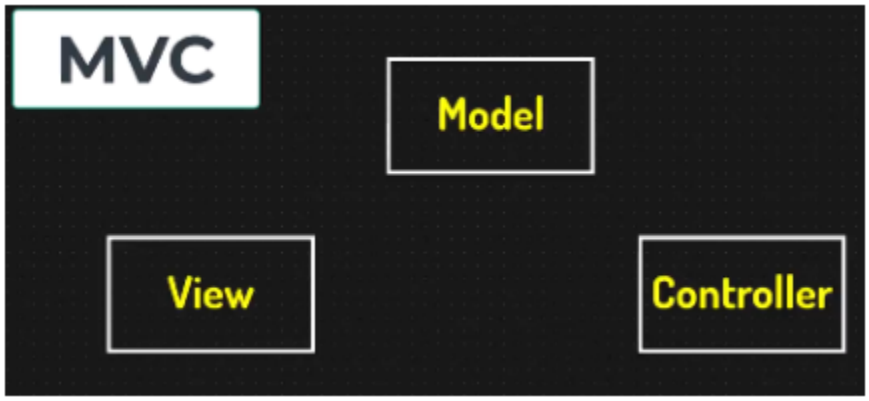
- **Purpose:** Primarily for creating dynamic web content by embedding Java code into HTML pages.
- **Syntax:** Combines HTML with Java code, using special tags (e.g., <% %> for scriptlets, <%= %> for expressions).
- **Ease of Use:** More user-friendly for designing web pages as it separates the presentation logic from business logic.
- **Execution:** JSPs are compiled into servlets by the server at runtime, which then handles the request.
- **Maintenance:** Easier to maintain and update the look and feel as the logic and presentation are typically separated.
- **Use Case:** Ideal for view components and pages with mostly HTML content, where Java code is used minimally.

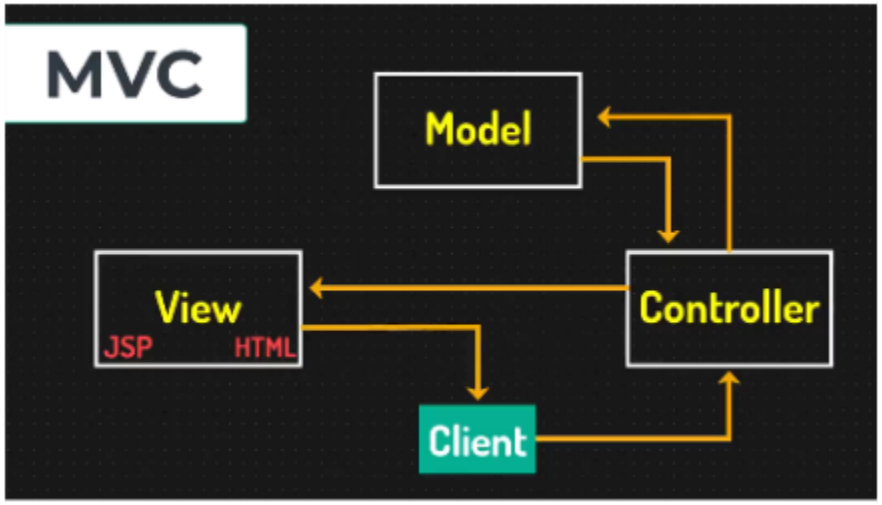
👉 Servlet:

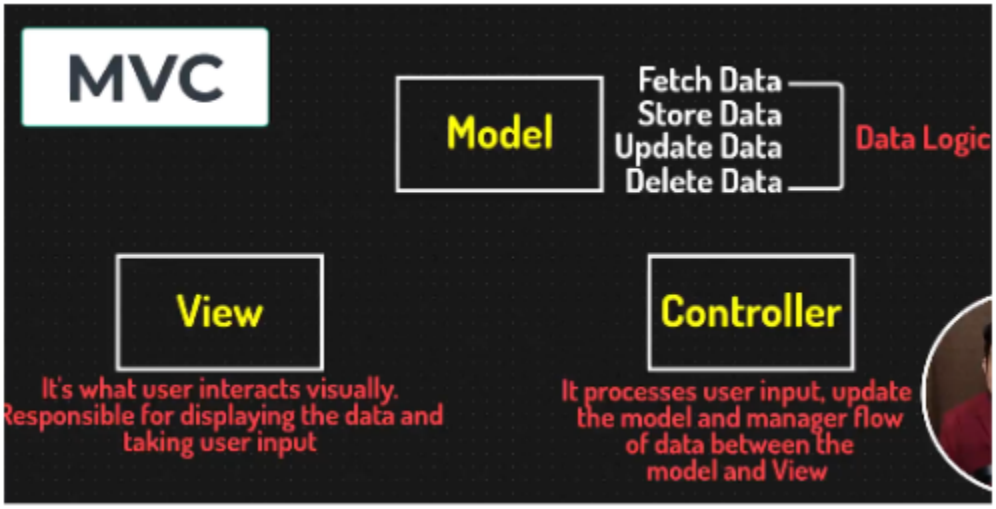
- **Purpose:** Used for handling HTTP requests and generating dynamic responses. They provide a programmatic way to create web applications.
- **Syntax:** Written in pure Java, handling both request processing and response generation within a single class.
- **Ease of Use:** More complex as it requires manual handling of HTTP requests, responses, and HTML generation.
- **Execution:** Servlets are Java classes that run on the server and are responsible for generating responses, usually returning HTML content.

- **Performance**: Servlets can be more efficient for scenarios requiring extensive server-side processing, as they avoid the overhead of JSP compilation.
- **Maintenance:** Can be more difficult to maintain due to intertwined request handling and HTML generation logic.
- **Use Case:** Suitable for complex business logic and handling tasks like database interactions, session management, and processing form data.

Introduction to MVC







👉MVC Overview:

- **Purpose:** The MVC pattern is used to separate an application into three interconnected components, helping to manage complex applications by dividing responsibilities and promoting organized code.

👉Components:

1. **Model:**
    - **Role:** Represents the data and business logic of the application.
    - **Responsibilities:**
        - Manages data retrieval, storage, and manipulation.
        - Notifies the view of any changes in the data.
        - Encapsulates the business rules and logic.
2. **View:**
    - **Role:** Responsible for presenting the data to the user and rendering the user interface.
    - **Responsibilities:**
        - Displays data received from the model.
        - Provides a way for users to interact with the application.
        - Updates the user interface in response to model changes.
3. **Controller:**
    - **Role:** Acts as an intermediary between the model and view, handling user input and updating the model.
    - **Responsibilities:**
        - Receives user input from the view.

- Processes input, performs actions (e.g., updating data), and interacts with the model.
- Updates the view based on changes to the model or user actions.

👉Workflow:

1. **User Interaction:** The user interacts with the view (e.g., submits a form).
2. Controller: The controller receives the input from the view, processes it, and interacts with the model.
3. **Model Update:** The model updates its state based on the controller's actions and may notify the view of changes.
4. **View Update:** The view retrieves the updated data from the model and refreshes the display for the user.

👉Benefits of MVC:

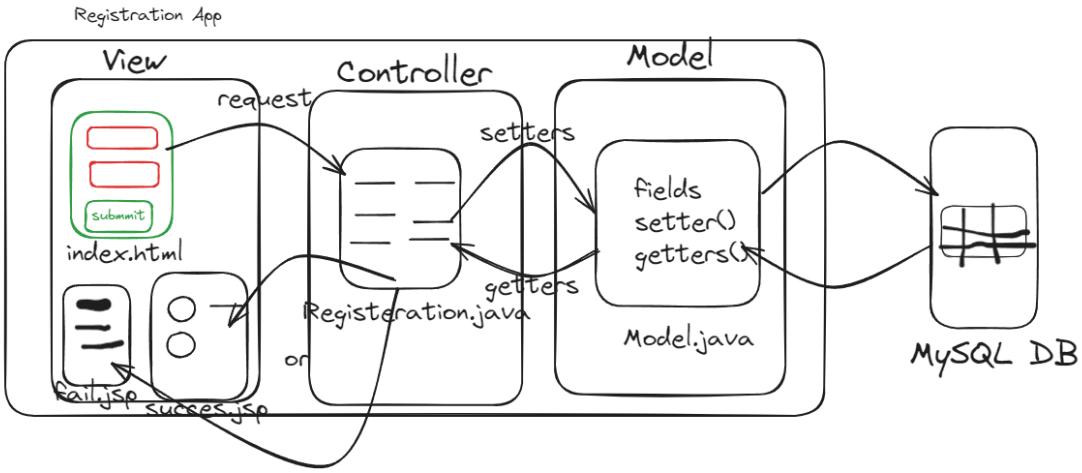
- **Separation of Concerns:** Each component has a distinct responsibility, making the system easier to manage, test, and scale.
- **Modularity:** Changes in one component (e.g., changing the view) have minimal impact on others (e.g., model or controller).
- **Maintainability:** Improved maintainability due to clear separation of business logic, data, and user interface.
- **Reusability:** Components can be reused across different parts of the application or in different projects.

👉Summary:

- Model handles data and business logic.

- View manages user interface and presentation.
- Controller processes user input and coordinates between the model and view.

MVC Using Servlet JSP and JDBC



👉Project Structure:

- MvcJspJdbcProject
  - Deployment Descriptor: MvcJspJdbcProject
  - JAX-WS Web Services
  - Java Resources
    - src/main/java
      - (default package)
        - JdbcUtil.java
        - RegisterController.java
        - RegisterDao.java
    - Libraries
  - build
  - src
    - main
      - java
      - webapp
        - META-INF
        - WEB-INF
        - failure.jsp
        - index.html
        - success.jsp

👉Model:

RegisterDao.java

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;
public class RegisterDao {
private String name;
private String email;
private String password;
private String city;
private Connection connection;
private PreparedStatement pstmt;
public String getName() {
	return name;
}
public void setName(String name) {
	this.name = name;
}
public String getEmail() {
	return email;
}
public void setEmail(String email) {
	this.email = email;
}
public String getPassword() {
	return password;
}
public void setPassword(String password) {
	this.password = password;
}
public String getCity() {
	return city;
}
public void setCity(String city) {
	this.city = city;
}
```

```
public int registerInfo(String name, String email, String password, String city)
throws SQLException {
connection= JdbcUtil.getConnection();
int rst=0;
if(connection!=null) {
        String query="Insert into personinfo (uname, email, upassword, ucity)
value(?,?,?,?)";
        pstmt=connection.prepareStatement(query);

        if(pstmt!=null) {
        pstmt.setString(1, name);
        pstmt.setString(2, email);
        pstmt.setString(3, password);
        pstmt.setString(4,city);

        rst= pstmt.executeUpdate();
        return rst;
        }
}
        return rst;
}
}
```

👉Controller:

RegisterController.java

```
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.sql.SQLException;
@WebServlet("/regst")
public class RegisterController extends HttpServlet {
```

```
	private static final long serialVersionUID = 1L;

	public void service(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
		 int rst=0;
		String name=request.getParameter("uname");
		String email=request.getParameter("uemail");
		String password=request.getParameter("upassword");
		String city=request.getParameter("ucity");
		RegisterDao dao=new RegisterDao();

		try {
		  rst=dao.registerInfo(name, email, password, city);

		} catch (SQLException e) {
			// TODO Auto-generated catch block
			e.printStackTrace();
		}

		if(rst!=0) {
			response.sendRedirect("success.jsp");
		}
		else {
			response.sendRedirect("failure.jsp");
		}
	}
}
```

👉Util library:

Jdbcutil.java

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
public class JdbcUtil {
private static Connection connection ;
```

```
	static {
		try {
			Class.forName("com.mysql.cj.jdbc.Driver");

			String url="jdbc:mysql://localhost:3306/telusko_db";
			String userName="root";
			String password="root";
			connection =DriverManager.getConnection(url,userName,
password);
		} catch (ClassNotFoundException e) {
			e.printStackTrace();
		} catch (SQLException e) {
			// TODO Auto-generated catch block
			e.printStackTrace();
		}
	}


	public static Connection getConnection() {
	return connection;
	}
}
```

👉View:

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>User Registration Form</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f4;
```

```
    margin: 0;
    padding: 0;
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
}
form {
    background-color: #fff;
    padding: 20px;
    border-radius: 8px;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
    width: 300px;
}
h2 {
    margin-bottom: 20px;
    font-size: 24px;
    text-align: center;
    color: #333;
}
label {
    margin-bottom: 10px;
    display: block;
    font-weight: bold;
    color: #555;
}
input[type="text"],
input[type="email"],
input[type="password"] {
    width: 100%;
    padding: 8px;
    margin-bottom: 15px;
    border: 1px solid #ccc;
    border-radius: 4px;
    box-sizing: border-box;
}
input[type="submit"] {
```

```
            width: 100%;
            padding: 10px;
            background-color: #28a745;
            border: none;
            border-radius: 4px;
            color: #fff;
            font-size: 16px;
            cursor: pointer;
        }
        input[type="submit"]:hover {
            background-color: #218838;
        }
    </style>
</head>
<body>
    <form action="./regst" method="POST">
        <h2>User Registration</h2>
        <label for="uname">Name:</label>
        <input type="text" id="uname" name="uname" maxlength="255"
required>
        <label for="email">Email:</label>
        <input type="email" id="email" name="uemail" maxlength="255"
required>
        <label for="upassword">Password:</label>
        <input type="password" id="upassword" name="upassword"
maxlength="255" required>
        <label for="ucity">City:</label>
        <input type="text" id="ucity" name="ucity" maxlength="255">
        <input type="submit" value="Register">
    </form>
</body>
</html>
```

Success.jsp

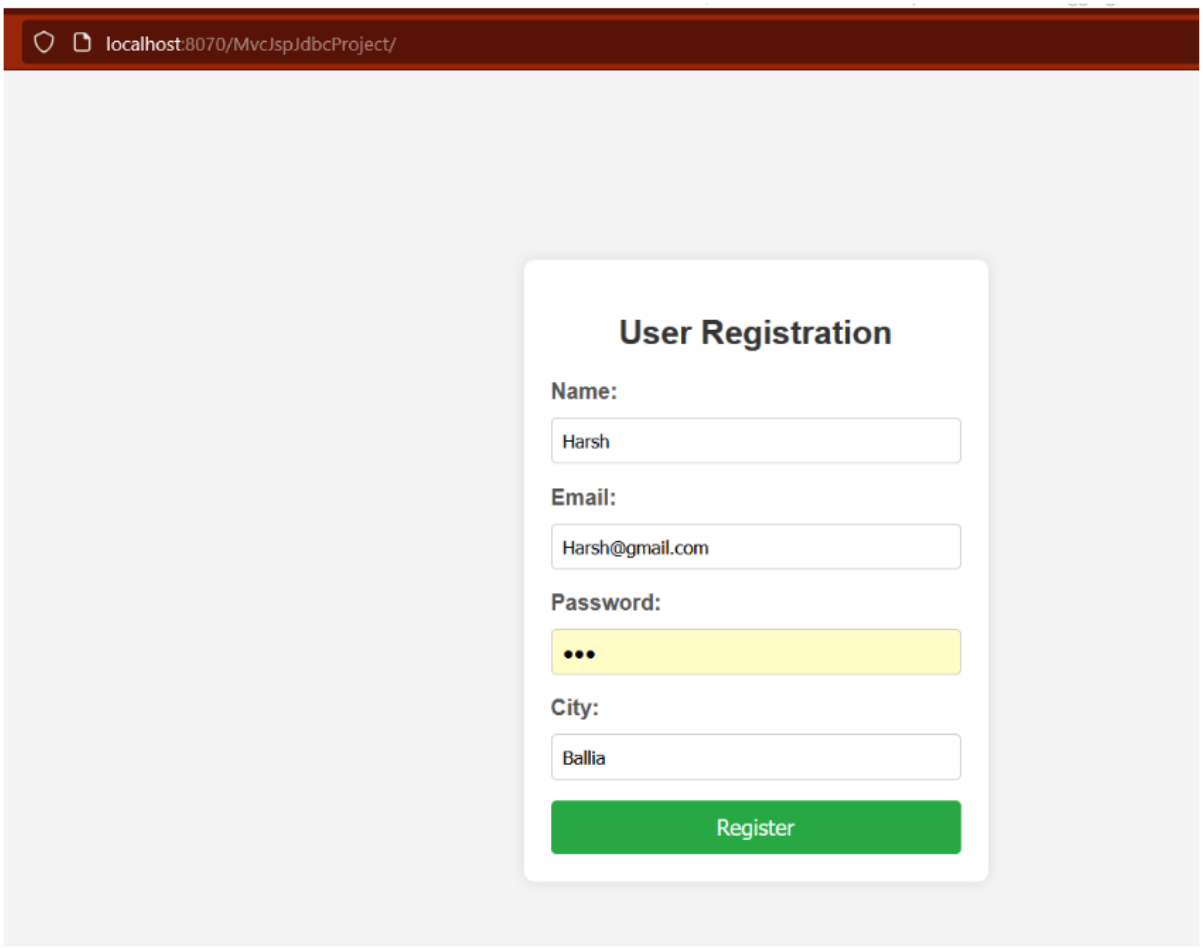
```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
```

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
<h1>Registration Succesful</h1>
</body>
</html>
```

Failure.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
   pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
<h1>Registration failure</h1>
</body>
</html>
```

👉Output:



Registration Succesful

```
mysql> select * from personinfo;
+-------+-----------------+-----------+--------+
| uname | email           | upassword | ucity  |
+-------+-----------------+-----------+--------+
| Shiva | ind@gmail.com   | 123       | ballia |
| Shiva | ind@gmail.com   | 123       | Ballia |
| Harsh | Harsh@gmail.com | 123       | Ballia |
+-------+-----------------+-----------+--------+
3 rows in set (0.00 sec)
```